

COMPX304-23A

Advanced Networking and Cybersecurity

Week 7
Introduction to Cyber Security

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Cyber Security

- This course is an overview of Cybersecurity
 - There are a lot of specialisations in the field
- Many solutions are dependent on risk appetite
 - More on this later



Cyber Security

- Background
 - Our systems are connected to networks
 - Can we assume they're operating within normal parameters?
 - Particularly, when adversaries are present?
- Key requirements (CIA):
 - Confidentiality
 - Integrity
 - Availability



Key Requirements: CIA

- Confidentiality
 - Information/functionality is selectively disclosed
 - Example Failure: The balance of an account is made available to the wrong person
- Integrity
 - Information/functionality is selectively modifiable
 - Example Failure: Someone else changes your account
- Availability
 - Information/functionality is available/usable to its intended users
 - Example Failure: Getting a timeout while trying to pay with EFTPOS



Terminology [ISO/IEC 27000]

- **Attack:** *“attempt to destroy, expose, alter, disable, steal or gain unauthorized access to or make unauthorized use of an asset”*
- **Threat:** *“potential cause of an unwanted incident, which can result in harm to a system or organization”*
- **Vulnerability:** *“**weakness of an asset or control** that can be exploited by one or more threats”*

Vulnerabilities are enumerated in Mitre’s CVE database

<https://www.iso.org/obp/ui/#iso:std:iso-iec:27000:ed-5:v1:en>



Additional Terminology

Weakness: *“A fundamental flaw, fault, bug, or other error in implementation, code, design, or architecture that could result give rise to a vulnerability.”*

For example:

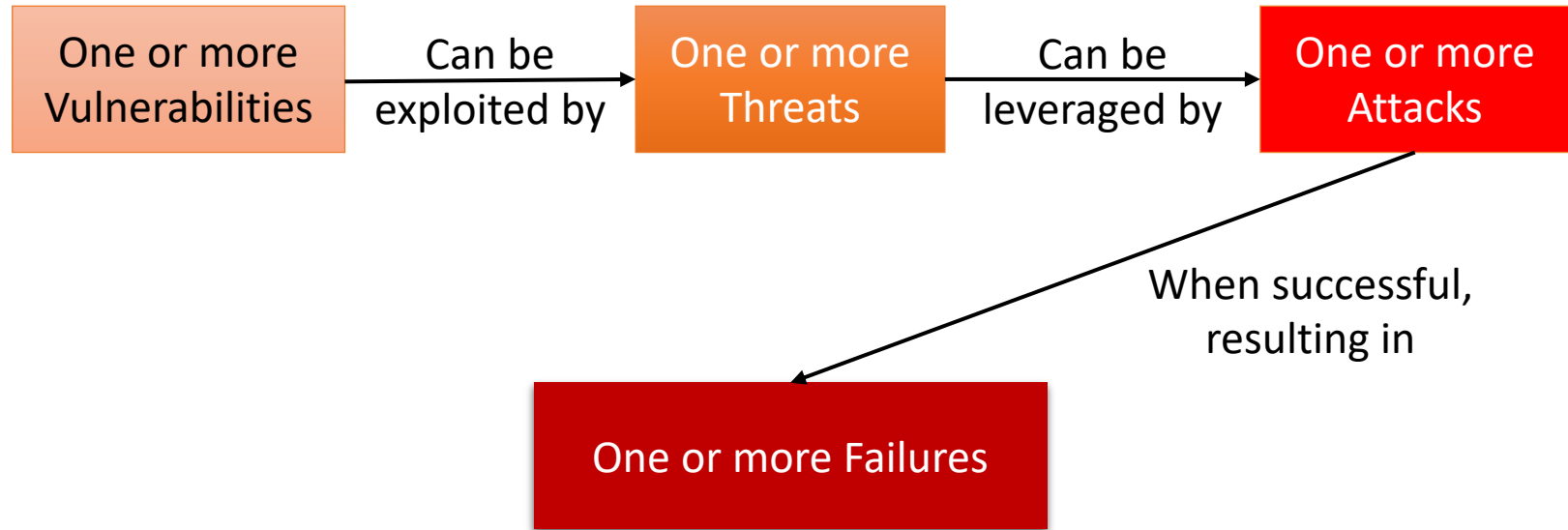
- Incorrect bounds checking
- Implicitly trusted input parameters

Not necessarily an issue, yet.

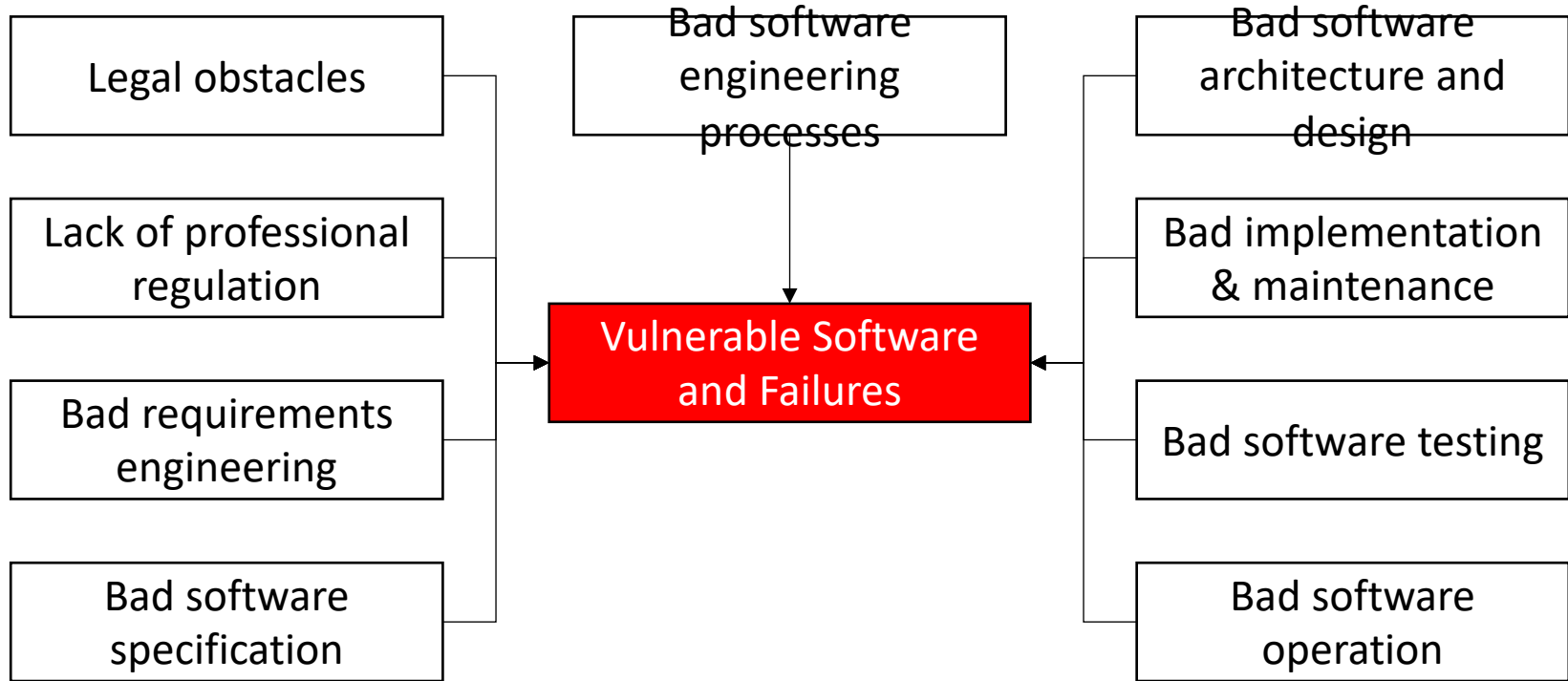
Enumerated in Mitre CWE database



Breaking it down



Common sources of vulnerabilities



Security = Minimising Risk

- Backups
 - Planned
 - Tested
- Incident Response
 - Who needs to be told
 - + more
- Procedural Guidelines
 - What needs done



Bird's eye view

Software applications

Operating System

Physical Hardware



Social Engineering

- Engineering is for the people
 - Which is sometimes forgotten...
 - Or taken advantage of
- Psychological manipulation
- People can be tricked
- Example
 - Donning a hi-vis vest, holding a hot beverage and a clipboard, and pressing a phone to your ear will get one into many places one shouldn't be.

<https://www.youtube.com/watch?v=NiEMcjSQOzg>



Social Engineering: Example Vulnerabilities

- People feel like
 - Returning a favour
 - Following through to a commitment
 - Acting similarly to others around them
 - Respecting authority figures
 - Accepting to do what people they like tell them to do
 - Taking advantage of opportunities
 - Rare or one-off
 - Discount



Social Engineering

- To reiterate and things to :
 - Urgency
 - Quid pro quo
 - Status Quo
 - Kindness
 - Authority



Professional Ethics!

- Remember!
- As professional Software Engineers and Computer Scientists
- You must not use this information to take advantage of people



Social Engineering: Example Attack Vectors

- Phishing
 - Legit appearing email to divulge personal information
 - E.g., redirects to a fake website where user logs in
 - **Spear phishing**: Highly customized to specific target
- Vishing
 - Similar but over the phone
- Smishing
 - Similar but via SMS
- Impersonation



Social Engineering: Example Attack Vectors

- Water holing
 - Set a trap in a usually visited website
 - People are relaxed vs. receiving a phishing email
- Baiting
 - Infected USB Keys, CDs, etc. with attractive titles
 - <https://www.youtube.com/watch?v=ZI5fvU5QKwQ>
- Quid pro quo
 - “I won’t need to wipe the phone to repair it if you give me the password”
- Tailgating/piggybacking
 - Run behind someone after they get access to a building



Social Engineering: Example Attack Vectors

- Shoulder surfing
 - Reading PINs or Passwords from people as they enter them
- Dumpster Diving
 - Literally as the name suggests
 - Documents that can be used to impersonate
 - Incorrectly disposed information
 - Unwiped or undestroyed storage



Professional Ethics!

- Remember!
- As professional Software Engineers and Computer Scientists
- You must not use this information to take advantage of people



Social Engineering: Countermeasure examples

- Don't open suspicious emails and attachments
- Multifactor authentication
- Don't trust tempting offers
- Don't give personal information online, over the phone, SMS,...
- Tell them you'll call them back and look up their number
- Be mindful of online-only friends
- Keep your software updated
- Maintain a cyber security awareness culture



Cyber attacks

- A large percentage (~80%) of cyberattacks start with social engineering.
 - Phishing
 - Spear Phishing
 - Dumpster diving etc.
- Usually the goal is to obtain a user's credentials to get into an enterprise network first. Once entry is gained attackers further exploit software and hardware vulnerabilities to gain privileges.



Social Engineering

- Exploits inherent vulnerabilities in humans
- Human vulnerabilities are harder to fix than software/hardware vulnerabilities
- How do we deal with vulnerabilities?
 - Find the vulnerability
 - Find the cause of the vulnerability
 - Create a solution to fix the vulnerability
- We often time have limited control over human weaknesses/vulnerabilities



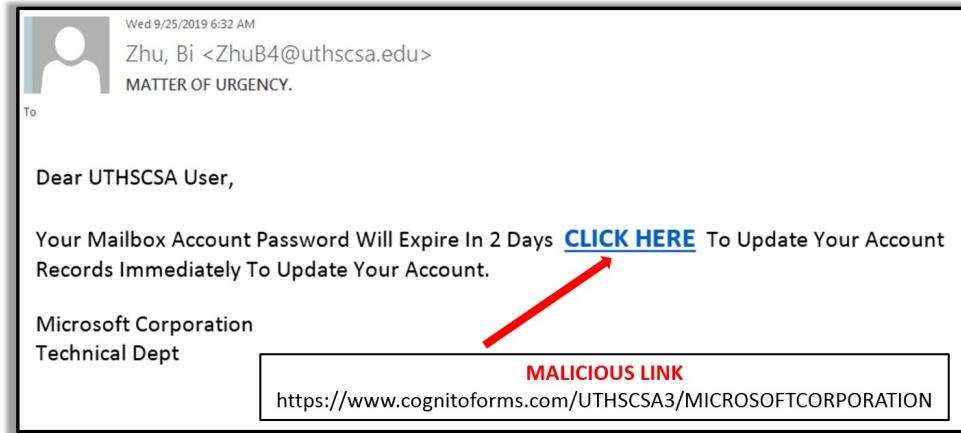
Social Engineering

- Uses psychology either explicitly or implicitly
- Example
 - Amygdala
 - Amygdala is a small part of our brain which processes and triggers strong emotions and the fight and flight response
 - Cortex
 - Cortex is the part of the brain that develops a logical response
 - In the event of a threat the Amygdala processes sensory information and prepares a response that may help a human evade the threat
 - The cortex takes times to process the information and develop a logical response

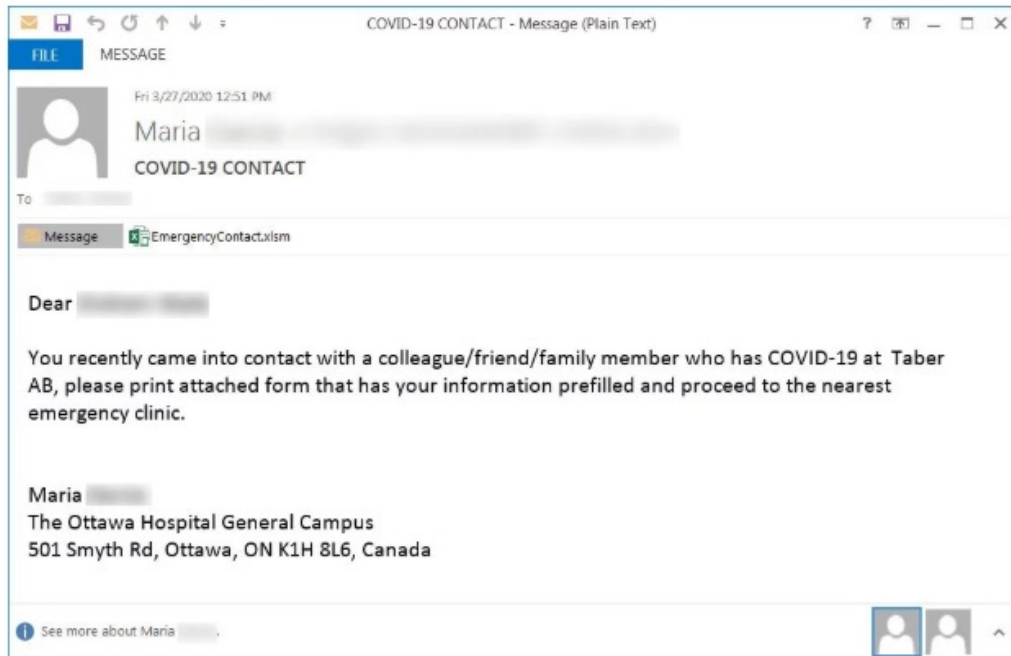


Urgency

- Attackers want the victim to take an action in a heightened emotional state



Fear

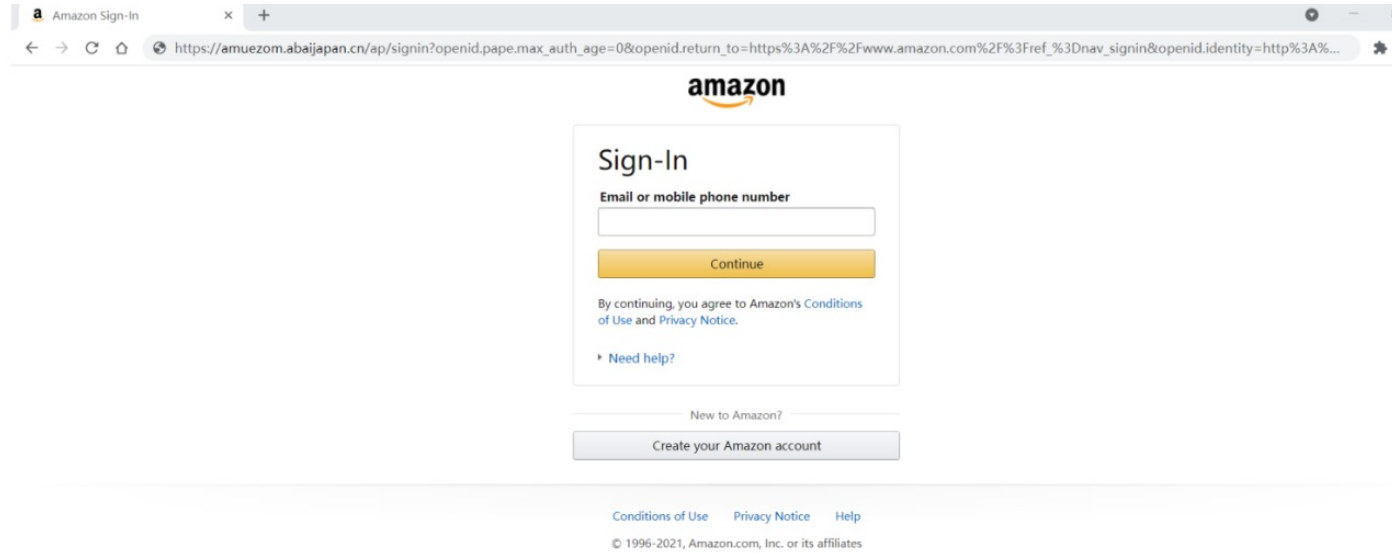


Curiosity



Social Engineering

- Attackers exploit trust and the need to feel trusted.



Social Engineering - Homographs

apple.com

apple.com



Social Engineering: Countermeasure examples

- Don't open suspicious emails and attachments
- Multifactor authentication
- Don't trust tempting offers
- Don't give personal information online, over the phone, SMS,...
- Tell them you'll call them back and look up their number
- Be mindful of online-only friends
- Keep your software updated
- Maintain a cyber security awareness culture



COMPX304-23A

Advanced Networking and Cybersecurity

Week 8
Application Security vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Who are the attackers?

- Nation states
 - US, China, Russia are prominent in the news as they engage in offensive operations but almost every country now has a cyber arm of military that conducts offensive operations
- Cybercriminals
 - Almost half of the crime in developed countries is cybercrime (by volume and by value)
 - Money from state-run cybercrime is almost 8% of the GDP of North Korea.
 - Cybercriminals attack vulnerable systems and compromise them to
 - Demand ransom
 - Use the systems for other attacks
- The Swamp (as Ross Anderson puts it)
 - Attacks for personal and sexual abuse, hate campaigns, cyber bullying, etc.



User input

- Any application that takes user input is prone to attacks
- When we write applications we assume the user will input the data in the way and form the application expects!!
- Writing an application to add two numbers
 - We expect user will give the application two **numbers**



Add two numbers and use the sum for division

[illegible]

Example runs

[illegible][illegible]

Injection attacks

- Information that is processed by computers can roughly be divided into two types
 - Data
 - Information that is processed
 - Instructions
 - Tell a computers how to process that data



Add two numbers and use the sum for division

[illegible]

Injection Attacks

- In our programs we usually expect
 - the user to provide the **data** while
 - the program code itself is a **combination of instructions and data**
- An injection attack is an attack where the **attacker manages to input instructions along with the input data**
- This results in the program doing something not intended by the programmer
- Injection attacks affect programs written in interpreted languages



SQL Injection

8.3M records of Freepik and Flaticon users stolen in SQL injection attack



BY DUNCAN RILEY

Data relating to 8.3 million users of stock-image sites Freepik and Flaticon, both owned by Freepik Co. S.L., have been stolen through an SQL injection attack.

The data stolen included the email addresses of users along with 3.77 million hashed passwords. Some 3.55 million of those passwords were encrypted with bcrypt, making them highly difficult but not impossible to crack, while 229,000 were salted MD5, an older encryption standard that can be easily decrypted. All users with the latter have had their passwords reset and affected users were sent an email encouraging them to change their password if it was used on another site.

<https://siliconangle.com/2020/08/24/8-3m-records-freepik-flaticon-users-stolen-sql-injection-attack/>



Asim Rahal, Independent Cloud Security Consultant
September 2, 2020

Share



Surging CMS attacks keep SQL injections on the radar during the next normal

Every year, millions of websites across the world fall victim to malware attacks that are designed to gain access to the site's backend without the administrator's knowledge in order to steal sensitive data or cause damage, usually for financial gain. This year, cyberattacks **have been on the rise** during the pandemic, leaving businesses to wonder whether or not things will settle down whenever the COVID-19 situation begins to wane, or if this is the next normal for the indefinite future.

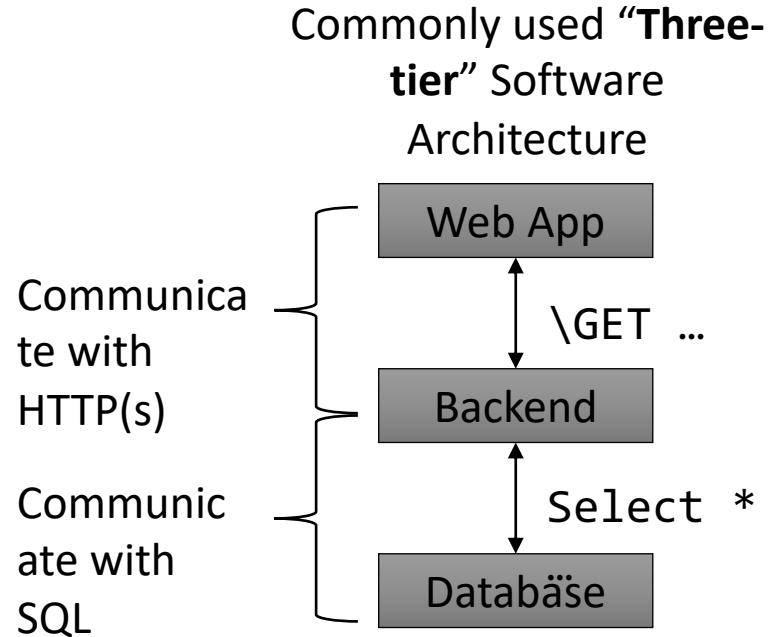
<https://www.helpnetsecurity.com/2020/09/02/surging-cms-attacks-keep-sql-injections-on-the-radar-during-the-next-normal/>



CROW Lab *Cybersecurity Researchers of Waikato*

SQL Injection

- Structured Query Language (SQL)
 - Used to send commands to a database
- SQL Injection
 - Malicious user injects their own SQL, which is executed as is by the DB



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

Expected Usage:

- User types a valid id, for example "5"
- The query finds the name of user with id 5



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

Way to attack the system:

- User types a more complicated string for id, for example:
""); DROP TABLE users;"
- This creates **two** queries one after the other
- The first query is badly formatted and it fails
- But, the second, separated by semi-colon, executes and ...



SQL Injection Example

```
read id;  
sql = "SELECT name FROM users WHERE id=" + id;  
db.runQuery(sql);
```

"\"; DROP TABLE users;" is a rather naïve attack...

How about:

```
"\"; INSERT INTO users(username, password, isAdmin) VALUES  
(\"newRoot\", \"\", TRUE);"
```

And the attacker just “rooted” the whole website!!



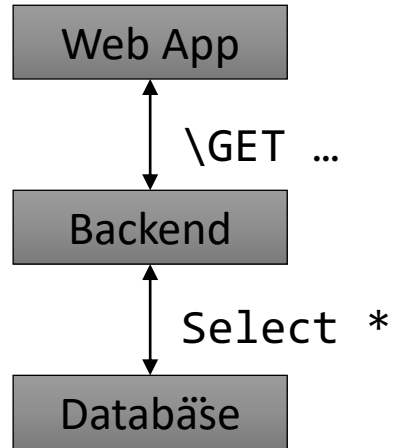
SQL Injection

- `SELECT count(name) FROM users WHERE name="{}`
 - Putting Vimal here works
 - So does `" or ""=""`; --

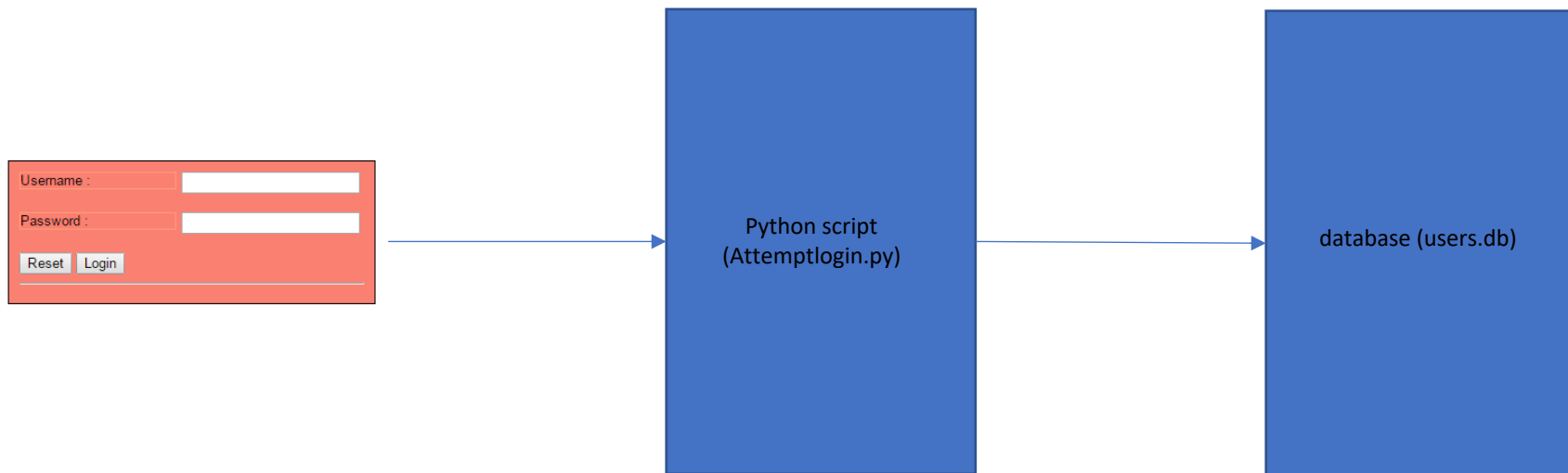


Three tier architecture

Commonly used “**Three-tier**” Software Architecture



Webpage with user input fields



`select * from users where username='{}' and password=x'{}'`

id	permissions	username	Password
0	1000	admin	ajkbhai2e3bjhbjh
1	500	bob	njbjkdnkj28829
2	10	alice	10nh7b8fff2ffffff3
3	5	root	128hbghiyvdi946



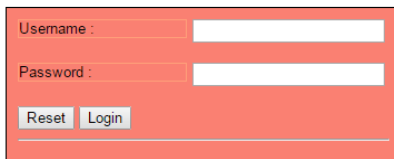
Login logic

- Correct username and password combination will select exactly one row
 - If no row is selected it means the username and password combination was incorrect
 - Appropriate action is taken in response to this such as letting the user know the username or password was wrong.
- If a row is selected as a result of the query
 - The username and password combination is correct and we can proceed with login
 - This usually transfers the user to a personalized page etc.
 - In the lab example, the user is showed the id and permission level.



Query examples

```
select * from users where username='{}' and password=x'{}'
```



Username :

Password :

```
select * from users where username='admin' and password=x'ajkbhai2e3bjhbjh'
```

```
select * from users where username='mario' and password=x'someword'
```



SQL injection

- The objective of sql injection is to inject sql query fragments as part of input data to
 - Make the query behave differently to what the programmer intended
 - In the case of a login page
 - The programmer intended the logic to work as previously described
 - Can we inject sql so that we can login without knowing the right username password combination

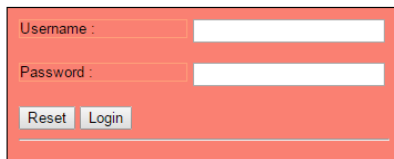


SQL injection

- One of the popular query fragments is

- ' OR ''=''; --

select * from users where username='{}' and password=x'{}'



Username :

Password :

select * from users where username= ' ' OR '' = '' ; -- ' and password=x'somegarbage'



```
select * from users where username= ' ' OR ' ' = ' ' ; -- ' and password=x'somegarbage'
```

```
select * from users where username= ' ' OR ' ' = ' ' ; -- ' and password=x'somegarbage'
```

```
select * from users where  
    username= ' ' OR ' ' = ' ' ;
```

```
select * from users ;
```



Parameterised queries

- Parameterised queries can mitigate sql injection to a large extent
 - Parameterised queries are compiled not interpreted
 - In parameterised queries the construction of a SQL statement containing user input is performed in two steps:
 1. Specify the query's structure, leaving placeholders for each item of user input.
 2. Specify the contents of each placeholder



Lab 7

```
select * from users where username = ? and password =x'{'
```

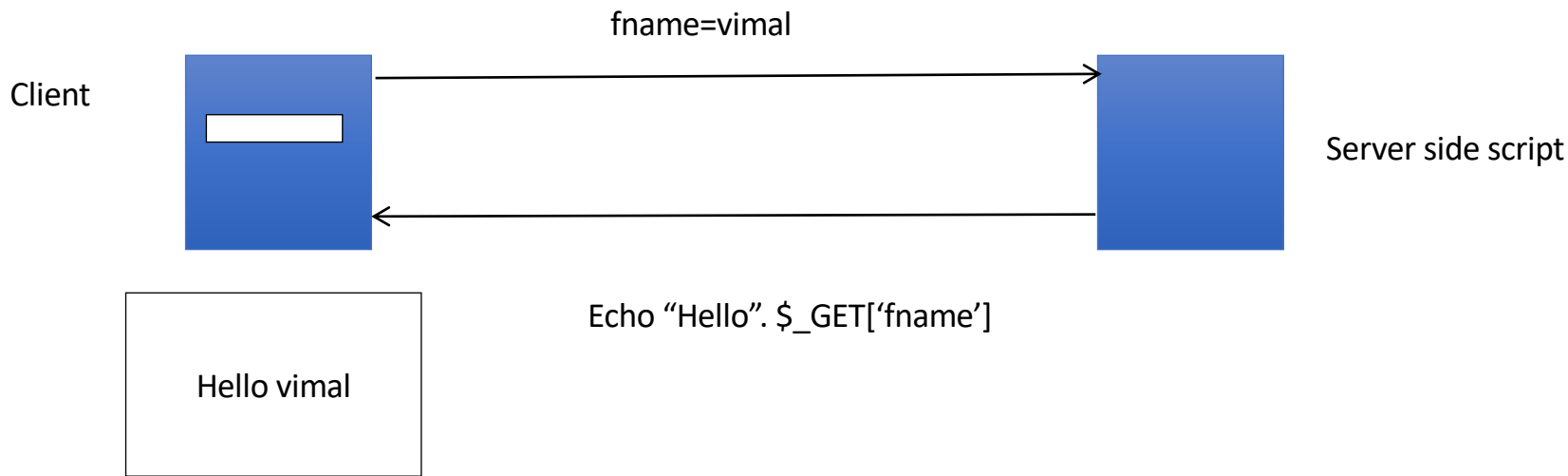
```
args = (user, )
```

user variable has previously been defined to be a string, so the input will be escaped to be a string



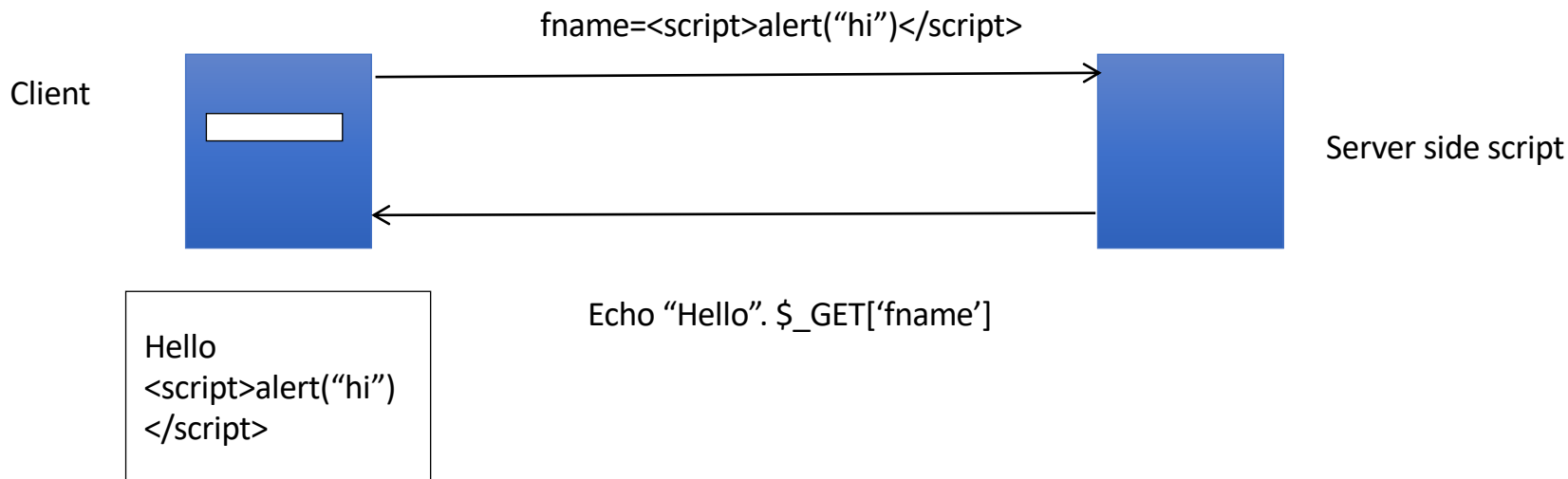
Cross Site Scripting

Cross-site scripting (XSS) is an injection attack that allows an attacker to execute malicious JavaScript in another user's browser



Cross Site Scripting

- If the attacker injects javascript code in the input field



How can an attacker embed JS in my URLs?

- Phishing (Social Engineering)
 - User receives a legitimate looking email with a URL which the email wants you to click on the pretext of
 - Account activation-deactivation/Shopping offers/Information from Bank/IRD/Police etc.
- URL shorteners are scary
 - <http://TinyURL/abcdef>
 - Who knows what is actually behind this URL



Malicious Javascript

- JavaScript has access to some of the user's sensitive information, such as cookies.
- JavaScript can send HTTP requests with arbitrary content to arbitrary destinations by using AJAX and other mechanisms.
- JavaScript can make arbitrary modifications to the HTML of the current page by using DOM manipulation methods.

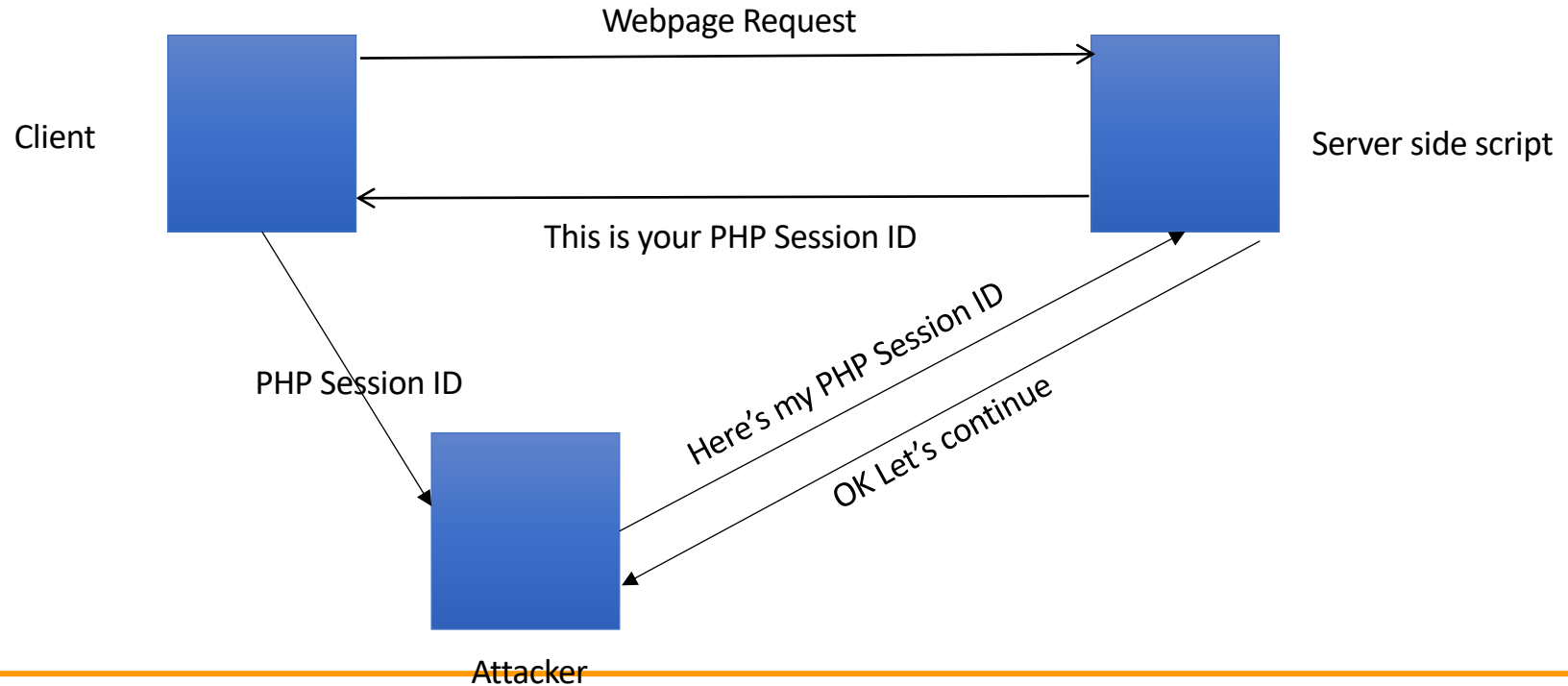


Malicious Javascript

- Cookie theft
 - The attacker can access the victim's cookies associated with the website using `document.cookie`, send them to his own server, and use them to extract sensitive information like session IDs.
- Keylogging
 - The attacker can register a keyboard event listener using `addEventListener` and then send all of the user's keystrokes to his own server, potentially recording sensitive information such as passwords and credit card numbers.
- Phishing
 - The attacker can insert a fake login form into the page using DOM manipulation, set the form's action attribute to target his own server, and then trick the user into submitting sensitive information.



Session Hijacking



COMPX304-23A

Advanced Networking and Cybersecurity

Week 9
Operating System Security/vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Stuxnet

An Unprecedented Look at Stuxnet, the World's First Digital Weapon

In an excerpt from her new book, "Countdown to Zero Day," WIRED's Kim Zetter describes the dark path the world's first digital weapon took to reach its target in Iran.



This recent undated satellite image provided by Space Imagery/Inta Space/Inta shows the once-secret Natanz nuclear complex in Natanz, Iran, about 150 miles south of Tehran. AP PHOTO/SPACE IMAGING/INTA SPACE/INTA



Stuxnet

- Targets industrial control systems
 - Such as those driving centrifuges of nuclear reactors
- Malicious behavior only if the right target is detected
- Rootkit to hide the other two components
- Propagated over an infected USB key



Stuxnet

- How could this happen?
- To answer this, we need to know a few more things:
 - Virtual Address Space Organization/Layout
 - Kernel mode/space vs. User mode/space
 - Operating System (OS) and User Interface
 - Computer Hardware & Device Drivers
 - Driver Signing

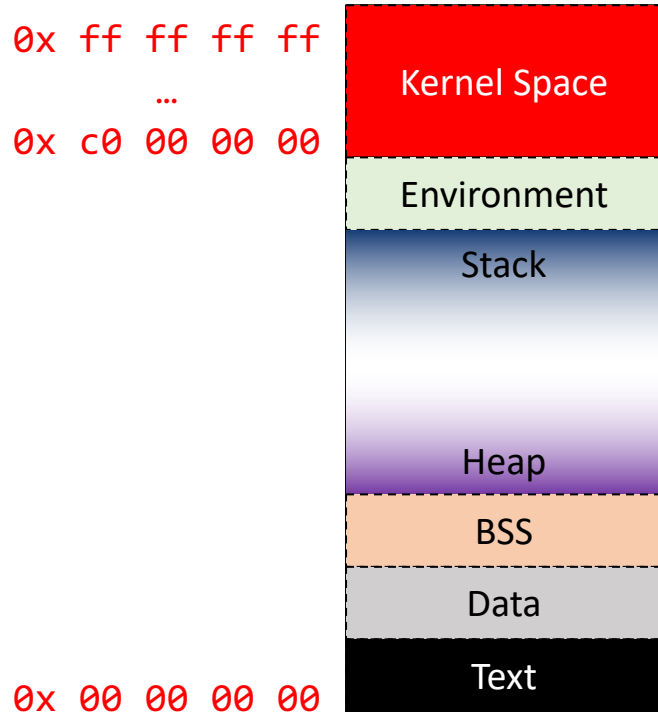


Operating system

Operating system manages and coordinates the use of hardware among the various application programs



Common Memory Layout of Programs

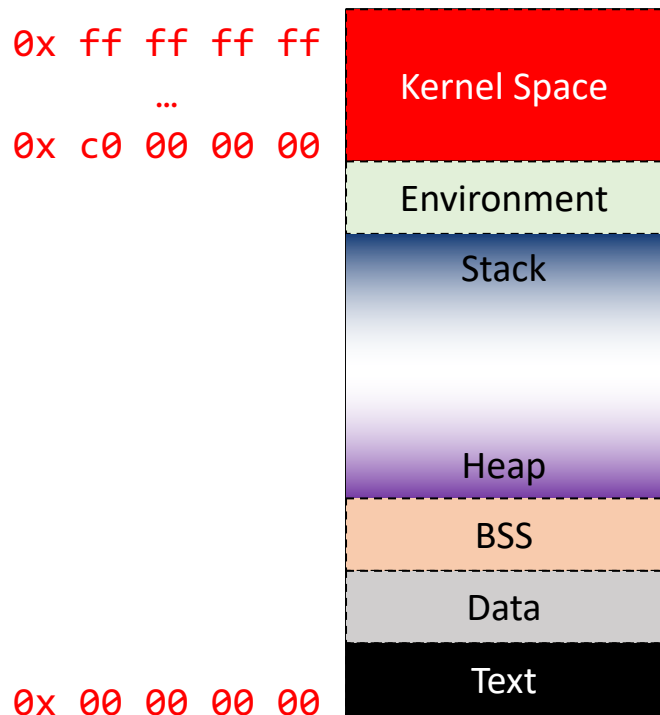


Kernel Space

- This is where the data and code of your OS lives
- Direct access not allowed



Common Memory Layout of Programs



Environment

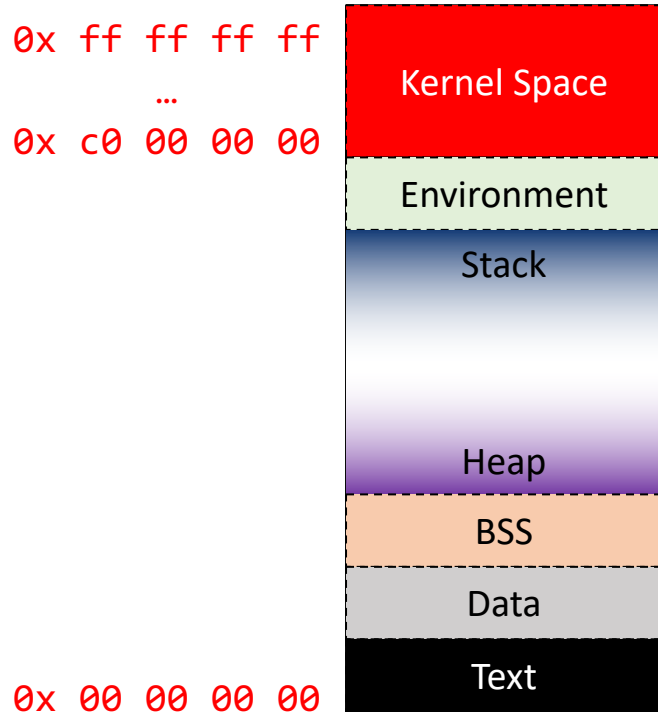
- Command-line arguments
- Environment Variables

```
int main(int argc, char *argv[]){...}
```

```
./myProg 1 2 hello
```



Common Memory Layout of Programs

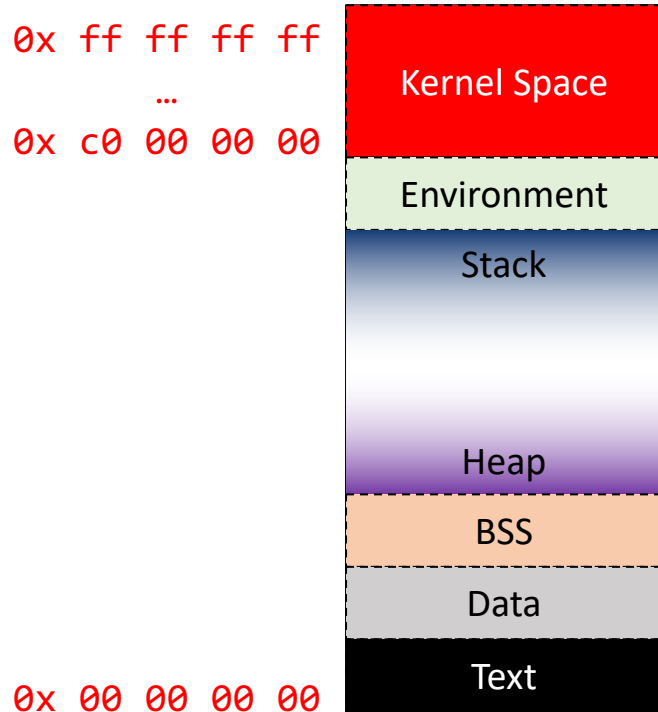


The Stack

- Comprises stack frames
 - One frame per invoked function
 - First one for main function
 - Local variables, return address, return stack pointer
- Stack frames are normally added downwards, towards lower addresses
- What might a stack overflow be?



Common Memory Layout of Programs

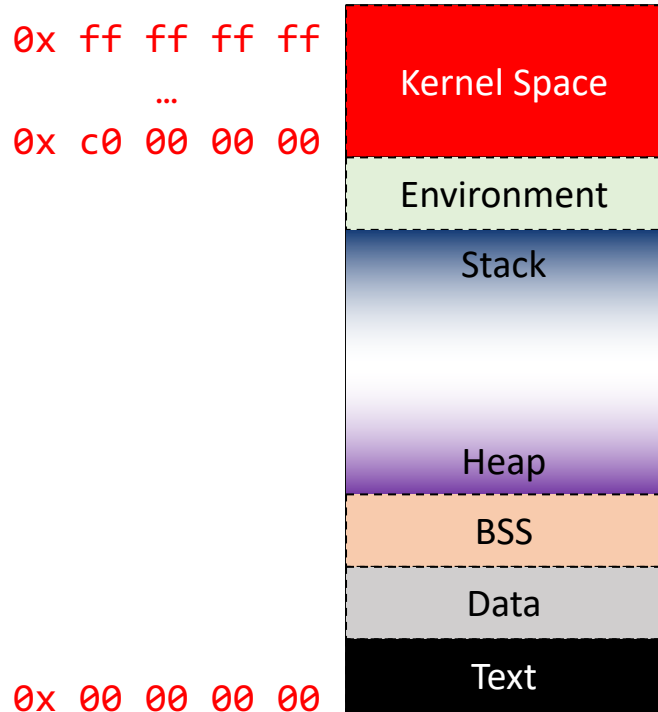


The Heap

- Manual memory management
- Memory chunks allocated by `malloc(size)`
- And are returned with `free(address)`
- What can happen if one allocated too many heap objects?



Common Memory Layout of Programs

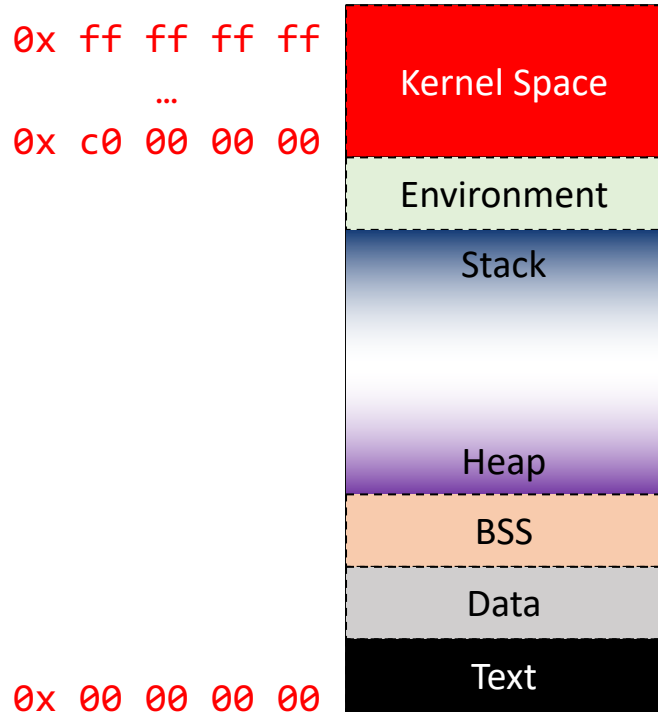


BSS

- Stores uninitialized variables/data
- Historical acronym (Block Started by Symbol)



Common Memory Layout of Programs

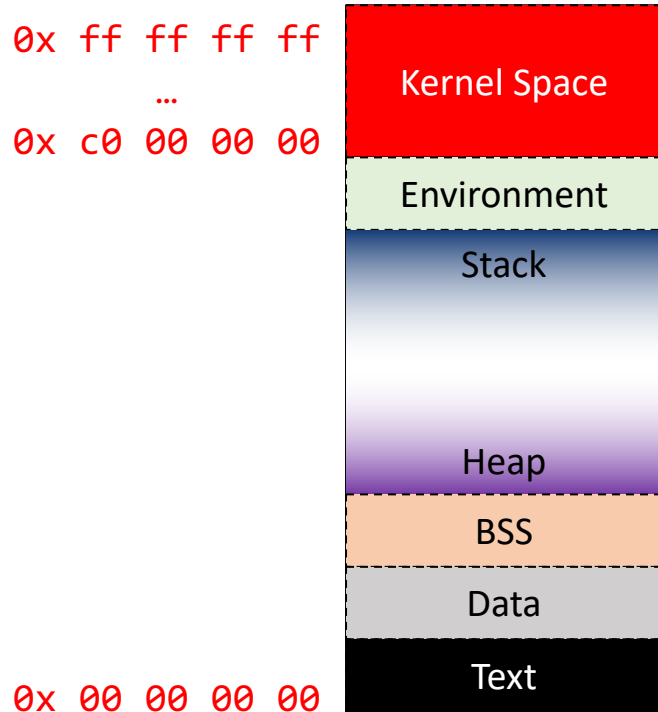


Data

- Stores initialized variables/data



Common Memory Layout of Programs



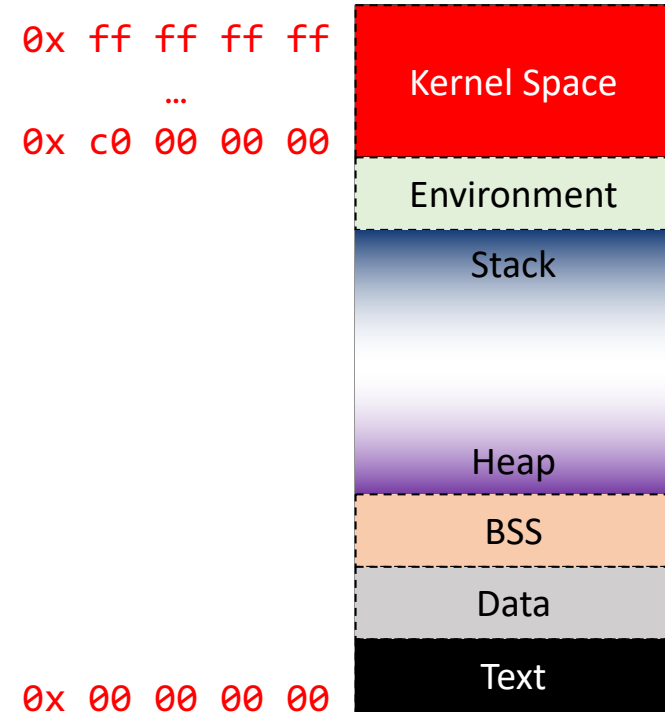
Text

- This is where your compiled code lives!



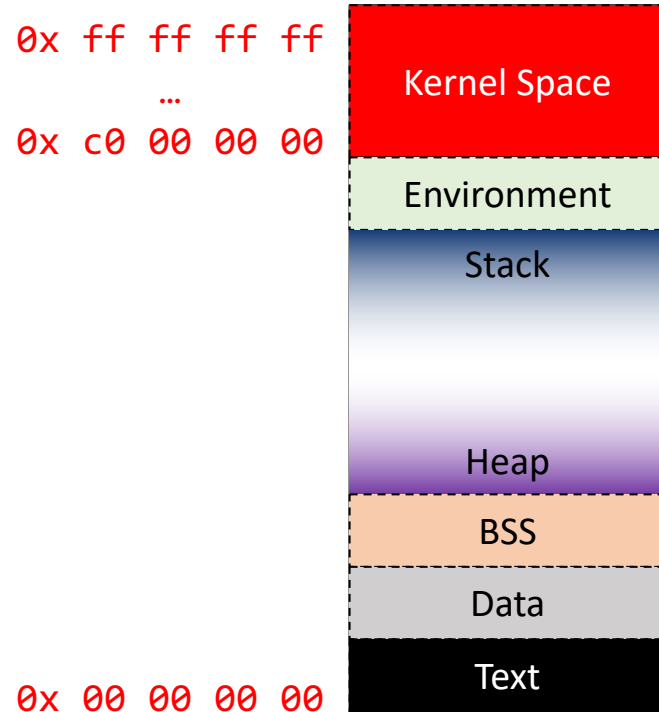
Kernel

- Core of the OS
- Complete control of the system
- Many responsibilities
 - Start up and boot
 - I/O requests
 - Manage tasks, memory, devices
 - Security
 - ...

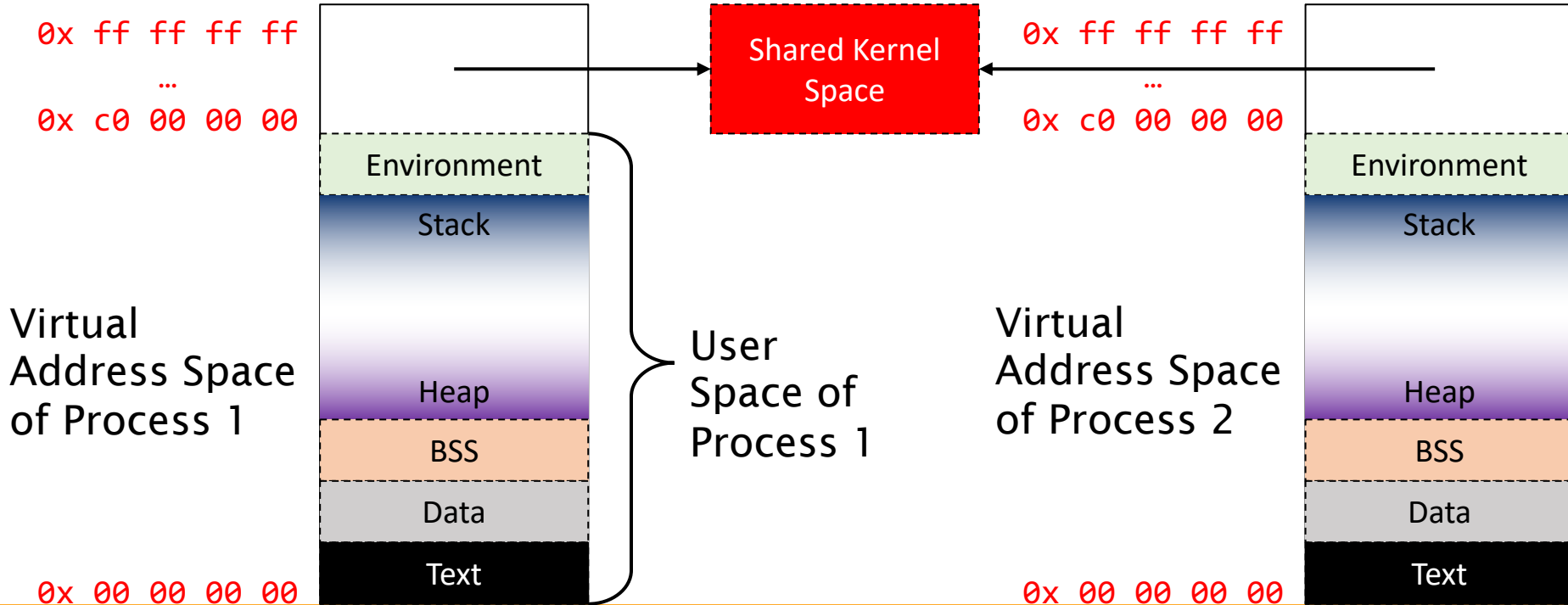


Kernel Space

- The Kernel needs its own memory to store all this
 - Kernel Space
- All running processes share the same kernel space
- But, have their own user space
 - The rest of the memory layout is user space



Kernel Space vs. User Space



CPU Modes and the Kernel

Security is enforced by the hardware!

Kernel Mode

- Any ISA instruction allowed
- Any I/O operation started
- Any memory area accessed

User Mode

- Some ISA instructions restricted
- No direct access to I/O
- Only a small area of the memory is accessible



Invoking the Kernel

- But how can the user invoke the kernel?
- User-mode doesn't have access to Kernel-space



Invoking the Kernel

- But how can the user invoke the kernel?
- Interrupts!
 - The flow of the application stops
 - Execution is transferred to special kernel-address
 - By the hardware! Security needs the hardware!!
 - The Interrupt handler decides what to do based on available information
 - Interrupt handler functions normally use the invoking user's memory space



Invoking the Kernel: System Calls

- An interrupt is too general
 - And doesn't have to be triggered by the user
 - E.g., may handle I/O
- Specific configuration needed for user calling the kernel:
 1. User prepares special registers with:
 - System call identification number
 - Parameters for the system call, including pointers to its user space
 2. User runs a software interrupt (or trap) ISA instruction
 3. Interrupt! Kernel-mode starts!
 4. Interrupt-handler realizes it's a system call
 5. It invokes it by looking at the special registers



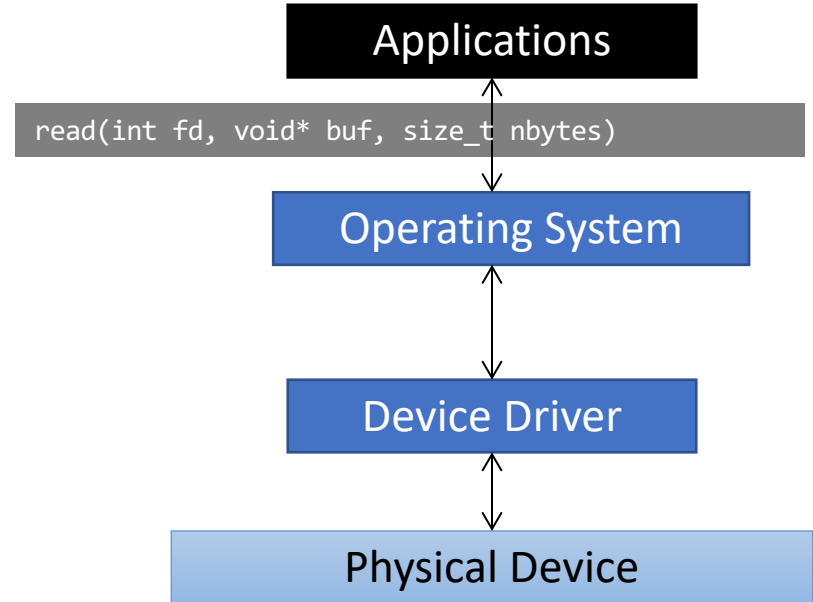
Kernel-space Information: EXTREMELY SENSITIVE!!!

- **Task Structures:** Tasks (processes, apps) activated
- **Process Table:** Points to Task Structures
- **Files and Inodes:** Open files
- **Memory management:** How the physical memory is allocated
- **Wait Queues:** What tasks wait on some resource
- **Kernel Semaphores:** Synchronize the tasks of the kernel
- **System Timers:** Used for scheduling, etc.



Computer Hardware and the OS

- Everything is a file in Linux
- The OS offers certain system calls:
 - Read, Write, Open, Close, Etc.
- Interacting with a device is not the same for all devices
- Each device needs to be driven according to its own specs
- Device Drivers!



Device Drivers

- Abstract control of a device
- Device-dependent
- Operating-System-specific
- Quote:

“Device drivers are a collection of routines which write magical numbers to magical places in the hardware”

- Security
 - Run in Kernel-Mode
 - Static or Dynamic Loading
 - Use Kernel-Space memory
- Otherwise
 - User apps would have direct access to:
 - See others' IO
 - Change others IO
 - Break/stall/etc. the device



Drivers

- Not all drivers are device drivers
- Some drivers do run in user-space
 - Really not common
- On Windows, GPU drivers are a special case
 - This is an aside 😊



But how can we just give access to a driver??

- Software running in kernel-mode can do:
 - ANYTHING!!
- Read and modify your data
- Install and/or start apps
- Uninstall and/or stop apps
- Crash your system (Unless it's a graphics driver)
- Wait and bide its time until something happens... ☹️



Driver Signing

- Association of a digital signature
- With a specific driver package
- A driver package must be verified before loaded into/by the kernel



Digital Signature

1. Alice encrypts her message with her private key
 $DS = \text{Encrypt}(M, A_{\text{private}})$
2. Alice sends both messages to Bob
 $\text{Send}(\text{Bob}, \langle M, DS \rangle)$

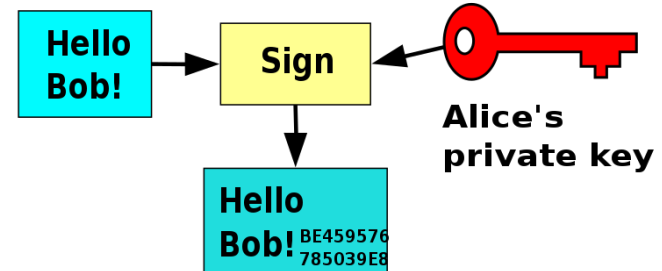
3. Bob decrypts the digital signature with Alice's public key

$$M' = \text{Decrypt}(DS, A_{\text{public}})$$

4. Bob compares the decrypted and openly submitted message to confirm authenticity

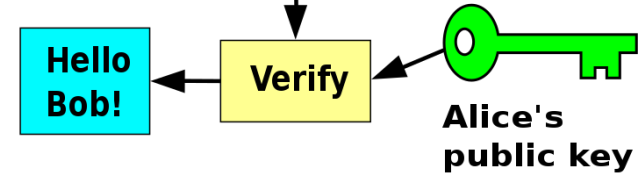
$$M = M'?$$

Alice



**Alice's
private key**

Bob



**Alice's
public key**

https://en.wikipedia.org/wiki/Digital_signature



Driver Signing

- Device manufacturers have private keys. They distribute the corresponding public keys to the Operating Systems manufacturers
- When a new device driver software is written the device manufacturer signs it with their private key
- If the Operating System is able to verify a software using one of the public keys it has, it will trust the software and use it



Stuxnet

- The hackers had stolen private keys from two reputable companies
 - Realtek and JMicron
- Used them to sign their malicious “drivers”
- The OS trusted them and installed them...!
- Once in the kernel:
 - It rapidly copied itself and propagated to other machines
 - Through other local-network vulnerabilities
 - It remained dormant unless it detected a specific Siemens control system
- Also, two websites were used to further control and patch the worm
 - Command and Control (C2)



COMPX304-23A

Advanced Networking and Cybersecurity

Week 10
Symmetric key Distribution + Hardware Vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



Symmetric Key Encryption

- Provides confidentiality
- Is faster compared to alternatives
- Is suitable for fast hardware implementation
- But
 - How do we exchange secret keys



Symmetric Key Distribution

- A can select a key and physically deliver it to B.
- A third party can select the key and physically deliver it to A and B.
 - Usually called a Key Distribution Center
- If A and B have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key.
- If A and B each has an encrypted connection to a third party C, C can deliver a key on the encrypted links to A and B.



Needham-Schroeder Protocol

- Third-party key distribution protocol
- For session between A B mediated by KDC protocol overview is:
 1. $A \rightarrow KDC: ID_A || ID_B || N_1$
 2. $KDC \rightarrow A: E_{K_a}[Ks || ID_B || N_1 || E_{K_b}[Ks || ID_A]]$
 3. $A \rightarrow B: E_{K_b}[Ks || ID_A]$



Needham-Schroeder Protocol

- used to securely distribute a new session key for communications between A & B
- but is vulnerable to a replay attack if an old session key has been compromised
 - then message 3 can be resent convincing B that is communicating with A
- modifications to address this require:
 - timestamps
 - using an extra nonce



Key Distribution Solutions

- Key distribution solutions based on symmetric key requires a KDC
- They also require the communicating parties to trust the KDC.
- There are key distribution solutions that use asymmetric cryptography that don't require KDCs and trust in KDCs but require public key infrastructure (PKI)
- The alternative (although expensive) is key exchange based on a quantum phenomenon
- Algorithms based on quantum computing can help perform key exchange without the use of KDCs and PKI.



Meltdown and Spectre

Intel's processors have a security bug and the fix could slow down PCs

By Tom Warren | @tomwarren | Jan 3, 2018, 8:45am EST

f t ↗ SHARE

<https://www.theverge.com/2018/1/3/16844630/intel-processor-security-flaw-bug-kernel-windows-linux>



SPECTRE



MELTDOWN

116

ANDY GREENBERG SECURITY 01.03.2018 03:00 PM

A Critical Intel Flaw Breaks Basic Security for Most Computers

A Google-led team of researchers has found a critical chip flaw that developers are scrambling to patch in millions of computers.

<https://www.wired.com/story/critical-intel-flaw-breaks-basic-security-for-most-computers/>

THIS IS HUGE. REALLY. —

Meltdown and Spectre: Here's what Intel, Apple, Microsoft, others are doing about it

Intel, Microsoft, ARM, and others have responded. We dig in.

PETER BRIGHT - 1/6/2018, 2:52 AM

<https://arstechnica.com/gadgets/2018/01/meltdown-and-spectre-heres-what-intel-apple-microsoft-others-are-doing-about-it/>

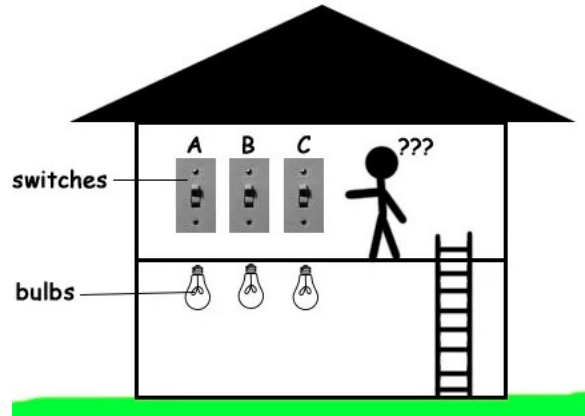


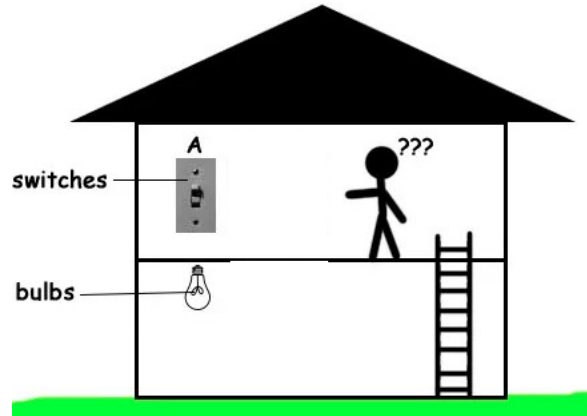
CROW Lab Cybersecurity Researchers of Waikato

Meltdown and Spectre

- Confidentiality attacks
- Extracts private information
 - Bit by bit
- Spectre
 - Breaks isolation
- Meltdown
 - Breaks isolation between user applications and the OS.







For a technical understanding

- we need to know about:
 - Computer Architecture
 - Computer Organization



Computer Architecture

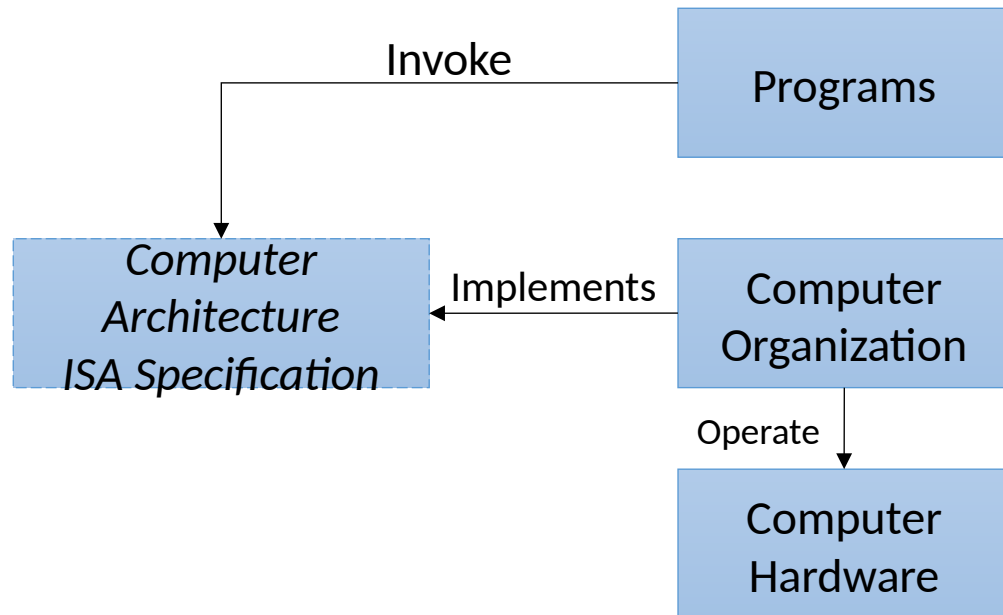
- The interface the CPU implements
- Also known as Instruction Set Architecture (ISA)
 - Load, store, add, etc.
- Programs execute in it. The CPU always repeats:
 - Fetch the next instruction
 - Decode it
 - Execute it

```
01 .MODEL SMALL
02 .STACK 100H
03 .CODE
04
05 MOV AX, 0x3C
06 MOV BX, 0000000000001010B
07 ADD AX, BX
08 MOV BX, 14
09 SUB AX, BX
10
11 MOV AH, 04CH
12 INT 21H
```



Computer Organization

- The mechanics under the ISA
- Transparent to the user
- But physical/real
- Not just a spec



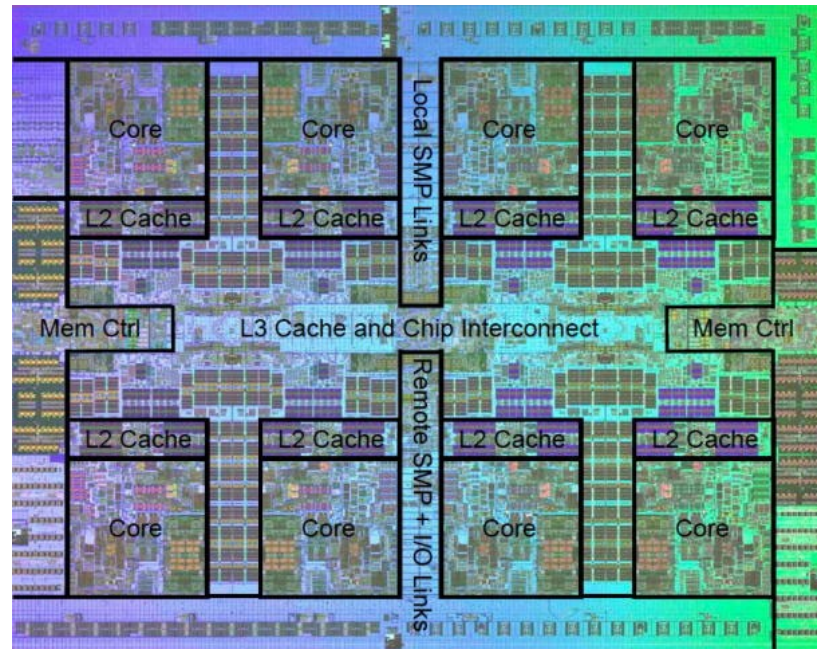
CPU Cache

- Main memory is much slower than the CPU
- We do not want the CPU to be idle for long periods of time
- Cache is a smaller but faster piece of memory closer to the CPU
- Every time something is fetched from the main memory it is also stored in the cache
- 80:20 rule



CPU Cache

- A load 0x... is not just a load!
 1. The address is converted to its physical address and permissions are checked
 2. The address is looked up in the Level-1 (L1) cache. If not there,
 3. It's looked up in the L2 cache. If not there,
 4. In the L3 cache, if not there
 5. From main memory (cache miss)
- Each of these pathways take a significantly different time (CPU cycles) to complete



<https://arstechnica.com/gadgets/2009/09/ibms-8-core-power7-twice-the-muscle-half-the-transistors/>

COMPX304-23A

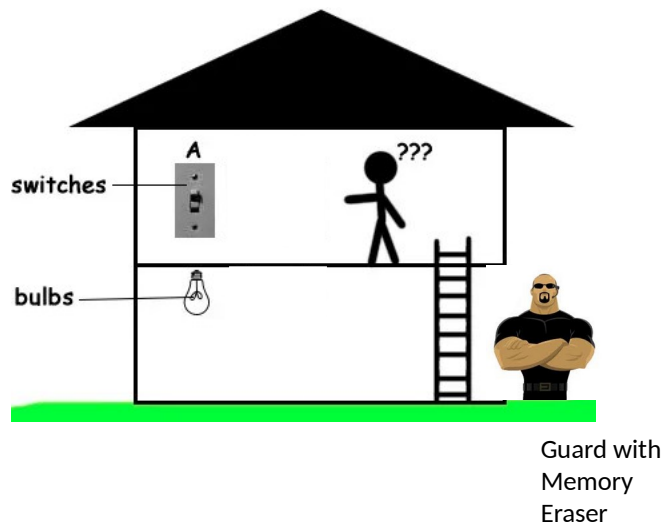
Advanced Networking and Cybersecurity

Week 11
Hardware Vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros





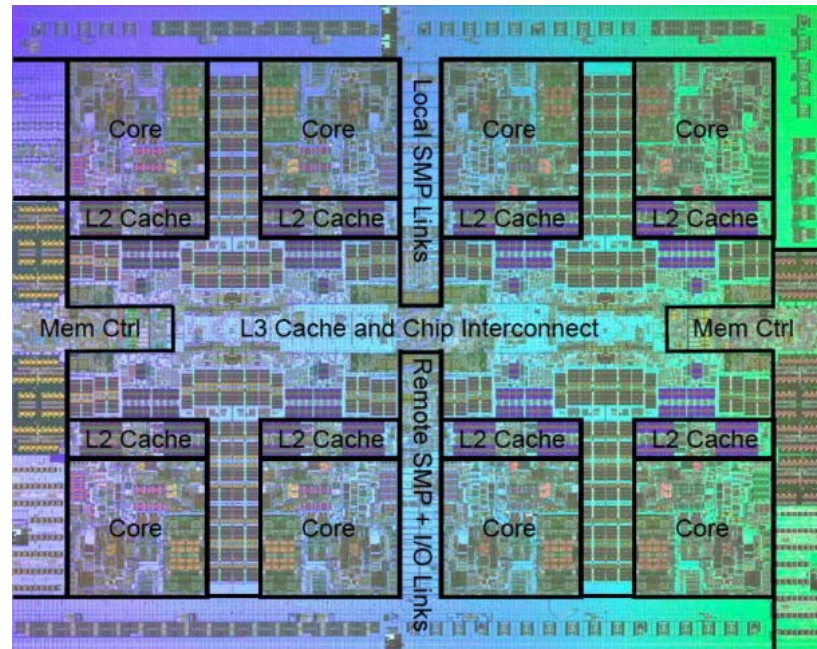
Side Channel attacks

- In channel attacks
 - Attacker uses the specified communication channel for attack
- Side channel attacks
 - The attacker uses unspecified input and output channels to convey system information
 - Examples
 - Timing
 - Disk Usage
 - Memory Usage
 - Sound
 - Power consumption
 - Cache Usage



CPU Cache

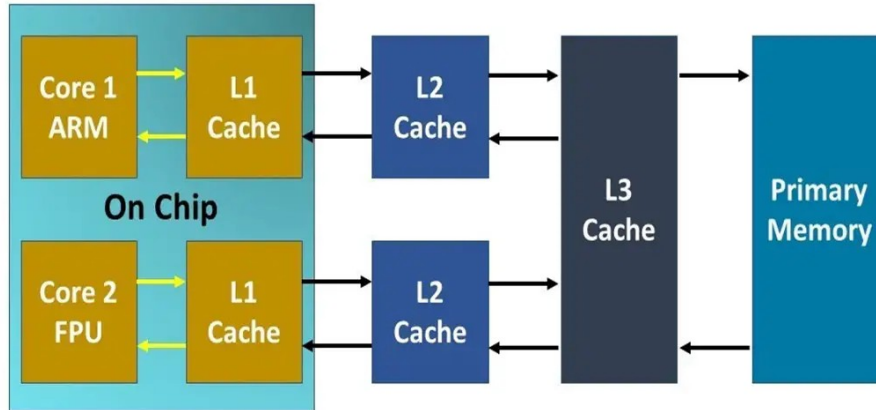
- A load 0x... is not just a load!
 1. The address is converted to its physical address and permissions are checked
 2. The address is looked up in the Level-1 (L1) cache. If not there,
 3. It's looked up in the L2 cache. If not there,
 4. In the L3 cache, if not there
 5. From main memory (cache miss)
- Each of these pathways take a significantly different time (CPU cycles) to complete



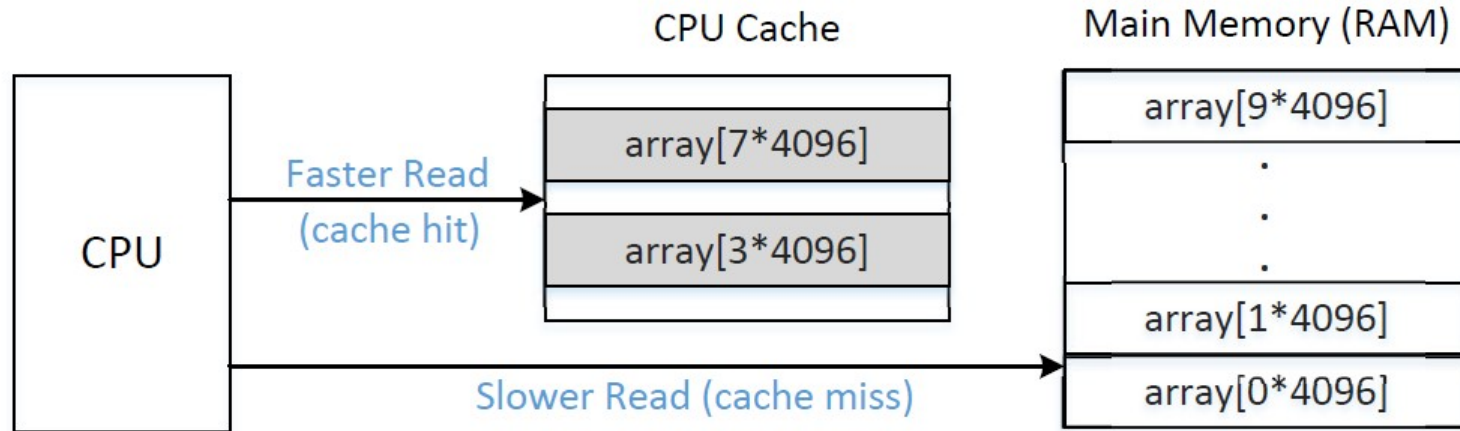
<https://arstechnica.com/gadgets/2009/09/ibms-8-core-power7-twice-the-muscle-half-the-transistors/>

CPU Cache

Cache Memory



CPU Cache



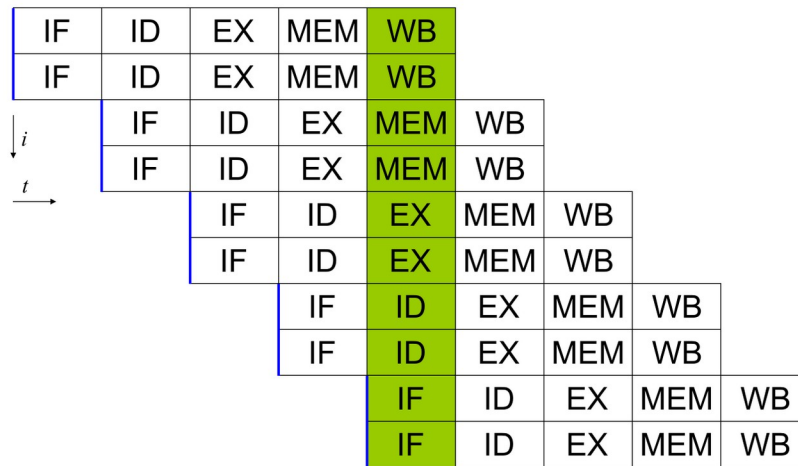
CPU Instructions (ISA)

```
01 .MODEL SMALL
02 .STACK 100H
03 .CODE
04
05 MOV AX, 0x3C
06 MOV BX, 00000000000001010B
07 ADD AX, BX
08 MOV BX, 14
09 SUB AX, BX
10
11 MOV AH, 04CH
12 INT 21H
```



Superscalar Execution (aka pipelining)

- ISA instructions are NOT executed one after the other!
 - Nor do they take one cycle to complete!
 - Instead, they are broken down to microarchitecture instructions
 - Each can be executed by a different component in the CPU
- Thus, we parallelize/pipeline them
 - What if there is a branch?



https://en.wikipedia.org/wiki/Superscalar_processor



Speculative Execution

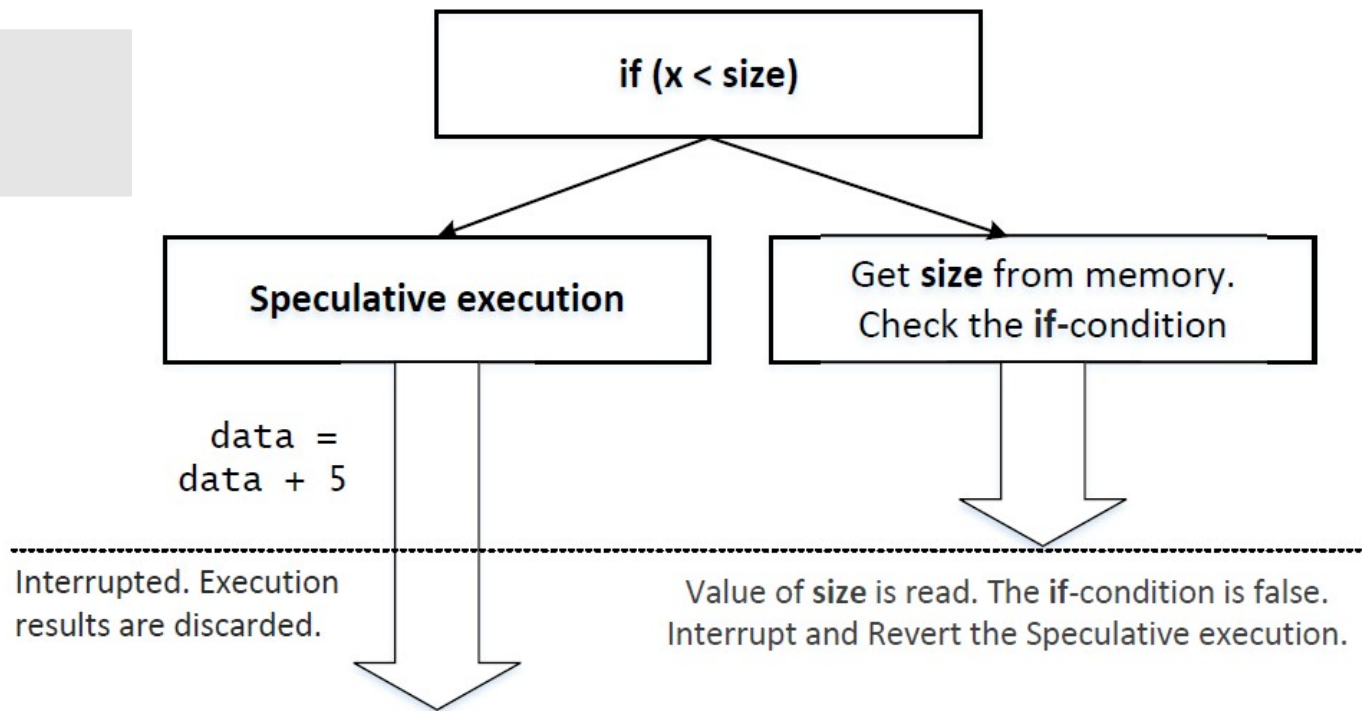
- The CPU guesses/learns the path the program will follow and starts executing it by putting it into the pipeline
- If it made a wrong guess, it flushes the pipe and starts from the right point undoing all changes it performed.

```
1 data = 0;
2 if (x < size) {
3     data = data + 5;
4 }
```



Speculative Execution

```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



Processor manufacturers forgot about the Cache

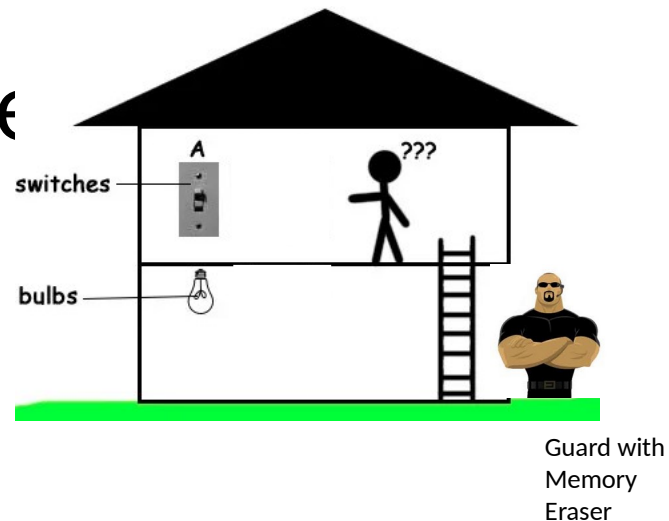


The Meltdown attack

- Attacker is a user program and is therefore in the user space.
- The attacker's motivation is to get hold of some confidential information in the kernel space.
 - The user space/kernel space separation using the on board hardware prevents an attacker from doing it directly.
 - The meltdown attack allows the attacker to do this using a side channel



From Lights to CPU Cache



Room = Kernel Memory

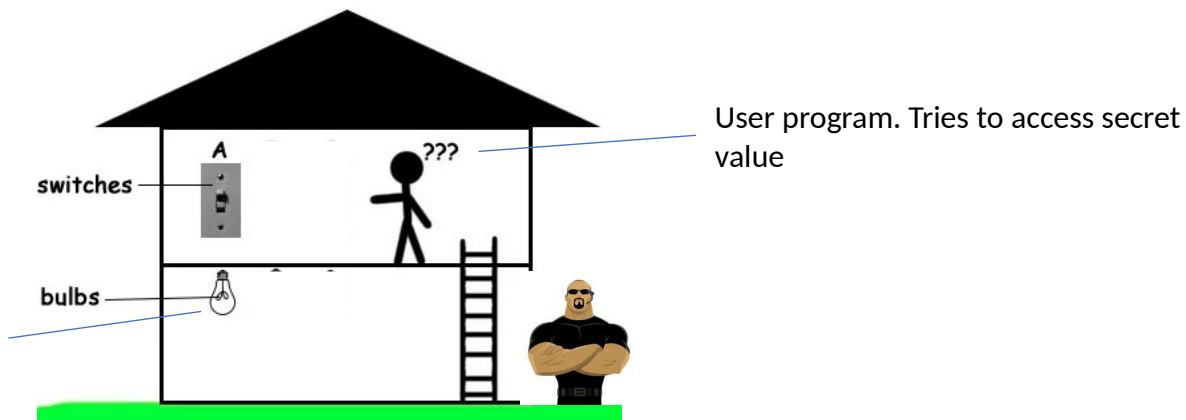
Secret number = Confidential kernel information

Guard = Access Control System. Uses the hardware to check if access is allowed.

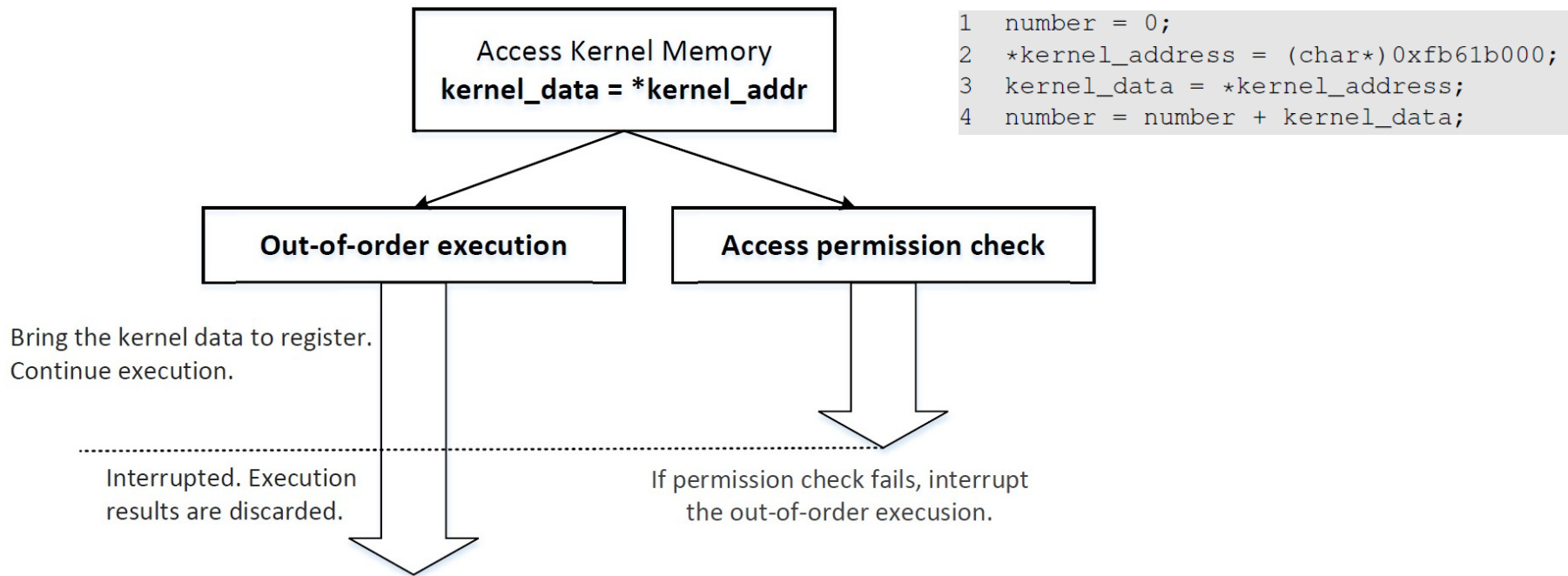


From Lights to CPU Cache

The information accessed by the program is now hot in the cache. The program can now read the cache and therefore the secret value



Speculative/Out-of-order Execution



Speculative/Out-of-order Execution

```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



The Spectre attack

- Makes use of speculative execution's drawback too
- Turns out you can (potentially) play this trick every time there is a condition in the code
- CPUs try to make an educated guess of which execution path is likely to be taken.
 - They do this by looking at which path was historically taken
- A CPU can be fooled into taking the incorrect execution path by training it with the correct execution path
- Eventually the CPU will correct itself but any data in the CPU cache as a result of the incorrect execution will not be erased



The Spectre attack

- Many access control decisions are taken using conditional code

```
if ( read_permission == 1)
    allow read
```

- Spectre class of attacks exploits speculative execution to get access to resources without having the required permission

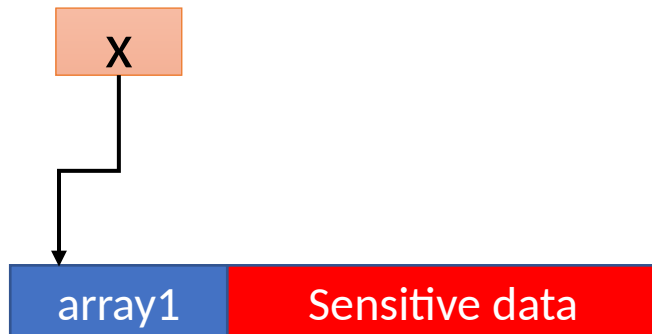


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

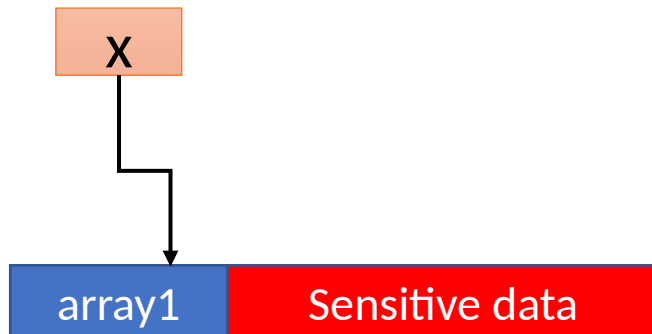


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

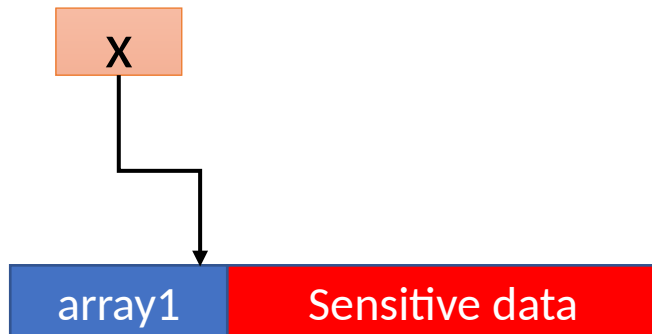


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

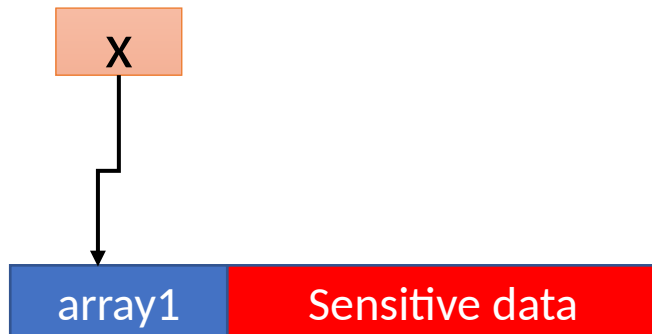


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

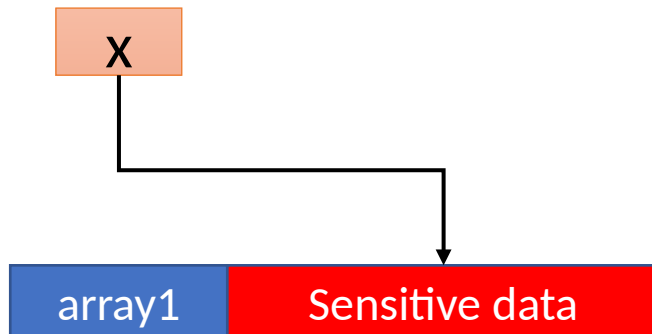
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview

Phase 2-a

Make x much larger than the length of the array and try to read beyond it!



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

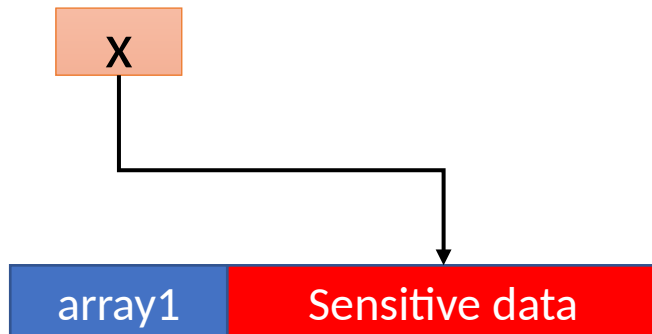
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview

Phase 2-b

The CPU is tricked and starts speculatively executing parts of the true-branch without having completed the actual check yet!



```
for(...) {  
    ...  
    → if (x < array1_size) {  
        → y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>



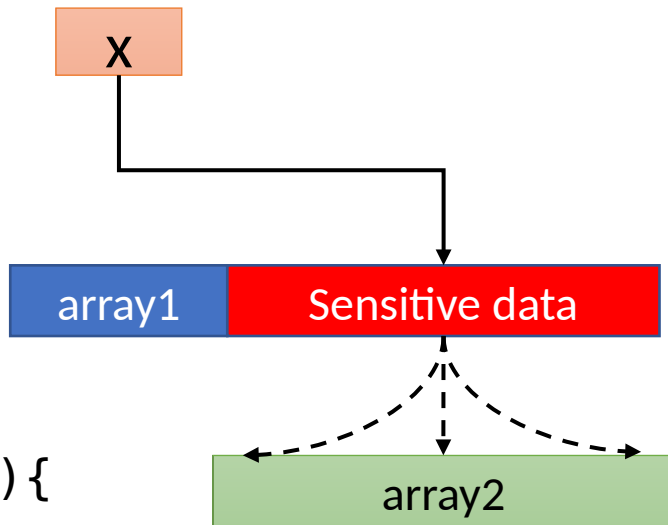
Spectre Phase 2-c

An index for array2 is
calculated:

$\text{array1}[x] * 4096$

Remember, $\text{array1}[x]$ is sensitive data now! The
index for array2 is sensitive too

E.g., if $\text{array1}[x]$ is 0, the
calculated index is 0. If it is 1,
the index is 4096. If 2, 8192
and so on...



```
for(...) {  
    ...  
    → if (x < array1_size) {  
        → y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

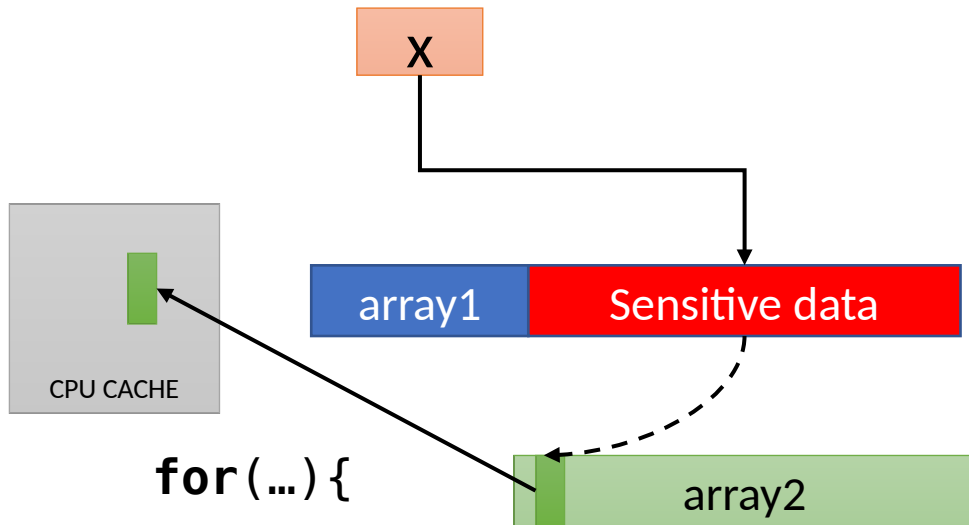


Spectre Overview

Phase 2-d

An index for array2 is calculated:
 $\text{array1}[x] * 4096$

The memory from array2 at that index is brought into the cache!



```
for (...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

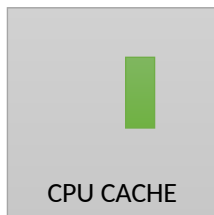


Spectre Overview Phase 2-e

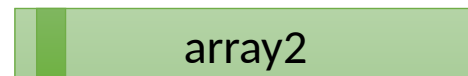
The CPU realizes its speculative execution failed and stops trying to execute the true-branch.

It unrolls all changes it performed.

But leaves the cache as is!!!



```
for (...) {
```



```
...  
    if (x < array1_size){  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```



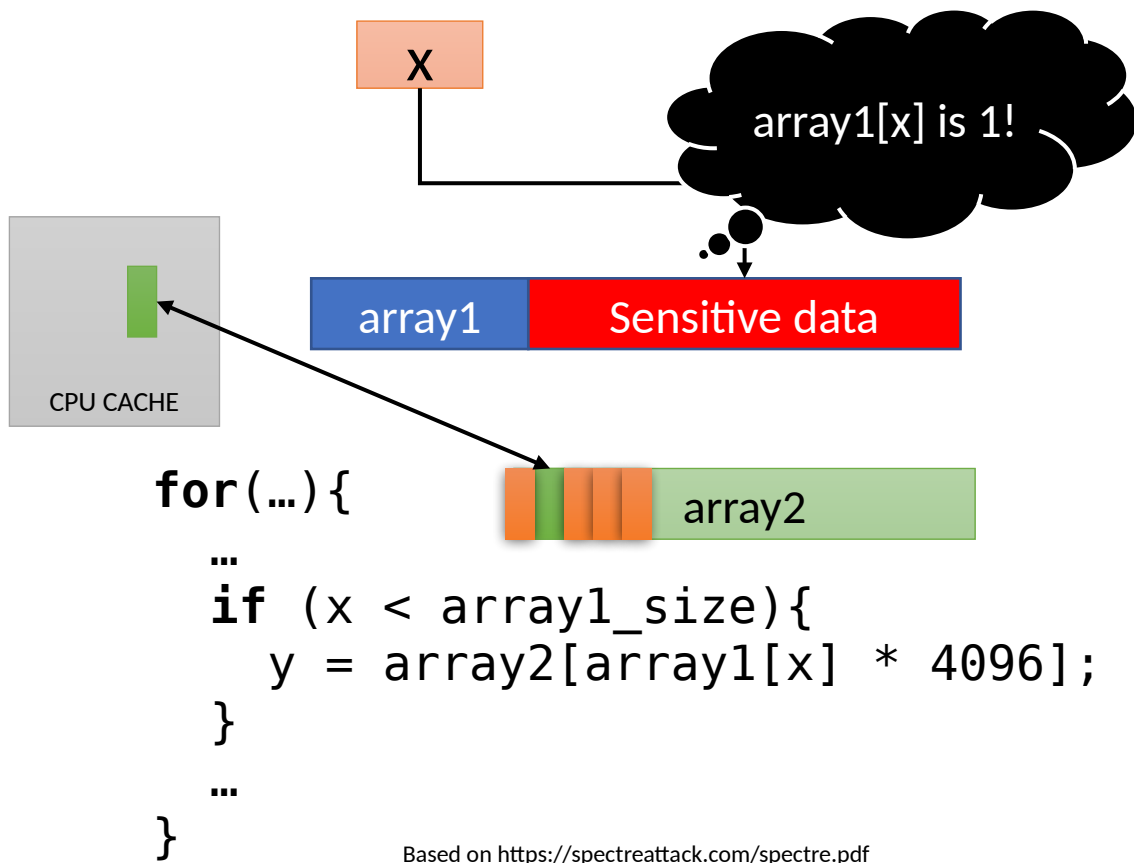
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview Phase 3

The attacker measures the time it takes to load a byte from array2 at a cache-line distance.

They will all be slow except one! The one already in the cache! Its position reveals the value of array1[x]



Based on <https://spectreattack.com/spectre.pdf>



Spectre Attacks: Exploiting Speculative Execution

Paul Kocher¹, Jann Horn², Anders Fogh³, Daniel Genkin⁴,
Daniel Gruss⁵, Werner Haas⁵, Mike Hamburg⁷, Moritz Lipp⁵,
Stefan Mangard⁵, Thomas Prescher⁶, Michael Schwarz⁵, Yuval Yarom⁸

¹ Independent (www.paulkocher.com), ² Google Project Zero,

³ G DATA Advanced Analytics, ⁴ University of Pennsylvania and University of Maryland,

⁵ Graz University of Technology, ⁶ Cyberus Technology,

⁷ Rambus, Cryptography Research Division, ⁸ University of Adelaide and Data61

Abstract—Modern processors use branch prediction and speculative execution to maximize performance. For example, if the destination of a branch depends on a memory value that is in the process of being read, CPUs will try to guess the destination and attempt to execute ahead. When the memory value finally arrives, the CPU either discards or commits the speculative computation. Speculative logic is unfaithful in how it executes, can access the victim's memory and registers, and can perform operations with measurable side effects.

Spectre attacks involve inducing a victim to speculatively perform operations that would not occur during correct program execution and which leak the victim's confidential information via a side channel to the adversary. This paper describes practical attacks that combine methodology from side channel attacks, fault attacks, and return-oriented programming that can read arbitrary memory from the victim's process. More broadly, the paper shows that speculative execution implementations violate the security assumptions underpinning numerous software security mechanisms, including operating system process separation, containerization, just-in-time (JIT) compilation, and countermeasures to cache timing and side-channel attacks. These attacks represent a serious threat to actual systems since vulnerable speculative execution capabilities are found in microprocessors from Intel, AMD, and ARM that are used in billions of devices.

While makeshift processor-specific countermeasures are possible in some cases, sound solutions will require fixes to processor designs as well as updates to instruction set architectures (ISAs) to give hardware architects and software developers a common understanding as to what computation state CPU implementations are (and are not) permitted to leak.

leverage hardware vulnerabilities to leak sensitive information. Attacks of the latter type include microarchitectural attacks exploiting cache timing [8, 30, 48, 52, 55, 69, 74], branch prediction history [1, 2], branch target buffers [14, 44] or open DRAM rows [56]. Software-based techniques have also been used to mount fault attacks that alter physical memory [39] or internal CPU values [65].

Several microarchitectural design techniques have facilitated the increase in processor speed over the past decades. One such advancement is speculative execution, which is widely used to increase performance and involves having the CPU guess likely future execution directions and prematurely execute instructions on these paths. More specifically, consider an example where the program's control flow depends on an uncached value located in external physical memory. As this memory is much slower than the CPU, it often takes several hundred clock cycles before the value becomes known. Rather than wasting these cycles by idling, the CPU attempts to guess the direction of control flow, saves a checkpoint of its register state, and proceeds to speculatively execute the program on the guessed path. When the value eventually arrives from memory, the CPU checks the correctness of its initial guess. If the guess was wrong, the CPU discards the incorrect speculative execution by reverting the register state back to the stored checkpoint, resulting in performance comparable to idling. However, if the guess was correct, the speculative execution

Kocher, Paul, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. "Spectre attacks: Exploiting speculative execution." *arXiv preprint arXiv:1801.01203* (2018).



COMPX304-23A

Advanced Networking and Cybersecurity

Week 11
Secure Software Development

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



What to do?

- We've seen examples of vulnerabilities and attacks
- Various levels
 - Users/People, Software, Operating Systems, Hardware
- Need for systematic approach for secure software
 - Secure Software Development and Architecture
 - Security Testing



Secure Coding Practices I

- Do not trust User Input
 - check nulls, types, reserved words, expected formats
- Don't ignore compiler warnings
- Keep it simple!
- Default deny

```
if isUserAuthorised:  
    authorizedAction()
```



```
if !isUserAuthorised:  
    return;  
  
authorizedAction()
```



Secure Coding Practices II

- Principle of least privilege
- Sanitize data sent to other systems
- Remember, multitier sanitization!!
 - Validate sanitization functions you use!

- People copy random code from the Internet... ☐

```
input = "<scr<script>ipt>nastyCode() </scr<script>ipt>"
```

```
print singleLevelSanitize(input)
```

```
<script>nastyCode()</script>
```

```
print multiLevelSanitize(input)
```

```
nastyCode()
```



Secure Coding Practices III

- Reuse existing **validated** modules, frameworks, ...
 - Don't write your own security code
 - Don't use low level languages unless needed
- Check for integer overflows
- Don't compare floats with strict equality:

```
float a, b;  
if(abs(a-b)>=threshold) //Good ☐  
if(a==b) //Bad ☐
```
- Check string lengths (buffer overflows)



Requirements

- Core CIA (Confidentiality, Integrity, Availability)
- Important extensions
 - Authentication
 - Identity verification
 - Nonrepudiation
 - Message ownership
 - Authorization
 - Privilege granting

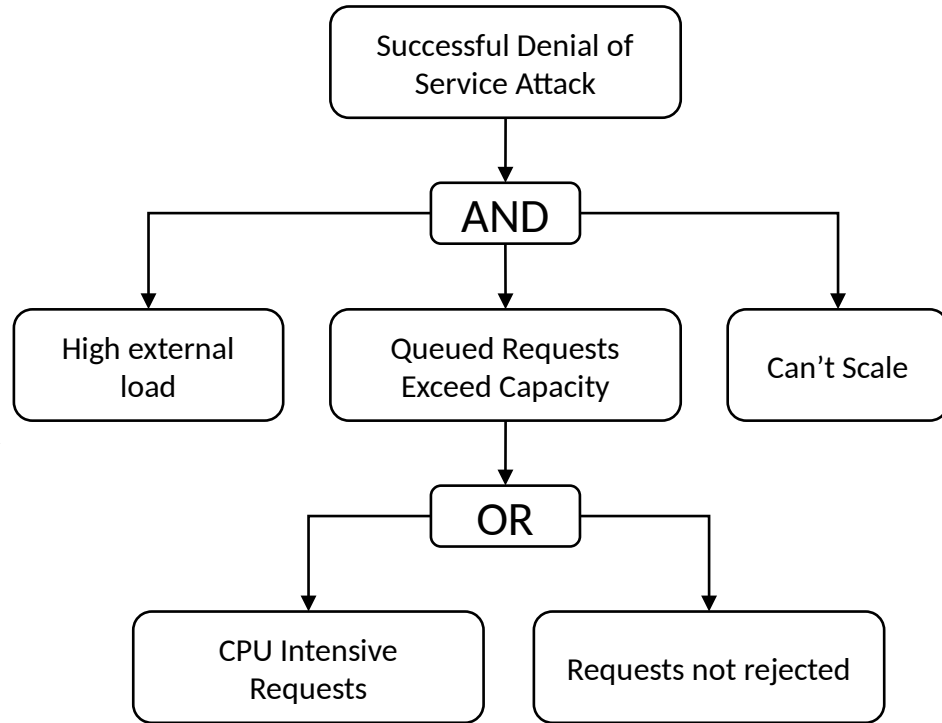


Threat Modelling

- Threat Modelling using attack trees

- Root
 - Successful Attack
- Nodes
 - Conditions required for the success of the attack
- Logical operators
 - Joining the conditions

- Allows one to think about how systems can be attacked and how one can prevent the conditions that enable an attack



Security Testing

- Effective QA
 - Test plans, security requirements & regression testing
 - Penetration tests
 - Source code auditing
 - Independent security reviews
 - Automated toolchains
 - Bug tracking tools
 - Secure coding standards

